

# Categorical Deep Learning: An Algebraic Theory of Architectures

Bruno Gavranović<sup>\*12</sup> Paul Lessard<sup>\*1</sup> Andrew Dudzik<sup>\*3</sup>  
 Tamara von Glehn<sup>3</sup> João G.M. Araújo<sup>3</sup> Petar Veličković<sup>3,4</sup>

## Abstract

We present our position on the elusive quest for a general-purpose framework for specifying and studying deep learning architectures. Our opinion is that the key attempts made so far lack a coherent bridge between specifying *constraints* which models must satisfy and specifying their *implementations*. Focusing on building a such a bridge, we propose to apply category theory—precisely, the universal *algebra of monads* valued in a 2-category of *parametric maps*—as a single theory elegantly subsuming both of these flavours of neural network design. To defend our position, we show how this theory recovers constraints induced by geometric deep learning, as well as implementations of many architectures drawn from the diverse landscape of neural networks, such as RNNs. We also illustrate how the theory naturally encodes many standard constructs in computer science and automata theory.

## 1. Introduction

One of the most coveted aims of deep learning theory is to provide a guiding *framework* from which all neural network architectures can be principally and usefully derived. Many elegant attempts have recently been made, offering frameworks to categorise or describe large swathes of deep learning architectures: Cohen et al. (2019); Xu et al. (2019); Bronstein et al. (2021); Chami et al. (2022); Papillon et al. (2023); Jogi et al. (2023); Weiler et al. (2023) to name a few.

We observe that there are, typically, two broad ways in which deep learning practitioners describe models. Firstly,

neural networks can be specified in a *top-down* manner, wherein models are described by the *constraints* they should satisfy (e.g. in order to respect the structure of the data they process). Alternatively, a *bottom-up* approach describes models by their *implementation*, i.e. the sequence of tensor operations required to perform their forward/backward pass.

### 1.1. Our opinion

It is our **opinion** that ample effort has already been given to both the top-down and bottom-up approaches *in isolation*, and that there hasn't been sufficiently expressive theory to address them both *simultaneously*. *If we want a general guiding framework for all of deep learning, this needs to change*. To substantiate our opinion, we survey a few ongoing efforts on both sides of the divide.

One of the most successful examples of the top-down framework is *geometric deep learning* (Bronstein et al., 2021, GDL), which uses a group- and representation-theoretic perspective to describe neural network layers via symmetry-preserving constraints. The actual realisations of such layers are derived by solving *equivariance constraints*.

GDL proved to be powerful: allowing, e.g., to cast *convolutional layers* as an exact solution to linear translation equivariance in grids (Fukushima et al., 1983; LeCun et al., 1998), and *message passing* and *self-attention* as instances of permutation equivariant learning over graphs (Gilmer et al., 2017; Vaswani et al., 2017). It also naturally extends to exotic domains such as *spheres* (Cohen et al., 2018), *meshes* (de Haan et al., 2020b) and *geometric graphs* (Fuchs et al., 2020). While this elegantly covers many architectures of practical interest, GDL also has inescapable constraints.

Firstly, usability of GDL principles to *implement* architectures directly correlates with how easy it is to resolve equivariance constraints. While PyG (Fey & Lenssen, 2019), DGL (Wang, 2019) and JGraph (Godwin et al., 2020) have had success for permutation-equivariant models, and e3nn (Geiger & Smidt, 2022) for E(3)-equivariant models, it is hard to replicate such success for areas where it is not known how to resolve equivariance constraints.

<sup>\*</sup>Equal contribution <sup>1</sup>Symbolica AI <sup>2</sup>University of Edinburgh  
<sup>3</sup>Google DeepMind <sup>4</sup>University of Cambridge. Correspondence to: Bruno Gavranović <bruno@brunogavranovic.com>, Paul Lessard <paul@symbolica.ai>, Andrew Dudzik <adudzik@google.com>, Petar Veličković <petarv@google.com>.

Because of its focus on groups, GDL is only able to represent equivariance to symmetries, but not all operations we may wish neural networks to align to are invertible (Worrall & Welling, 2019) or fully compositional (de Haan et al., 2020a). This is not a small collection of operations either; if we’d like to align a model to an arbitrary *algorithm* (Xu et al., 2019), it is fairly common for the target algorithm to irreversibly transform data, for example when performing any kind of a path-finding contraction (Dudzik & Veličković, 2022). Generally, in order to reason about alignment to constructs in **computer science**, we must go beyond GDL.

On the other hand, bottom-up frameworks are most commonly embodied in *automatic differentiation* packages, such as TensorFlow (Abadi et al., 2016), PyTorch (Paszke et al., 2019) and JAX (Bradbury et al., 2018). These frameworks have become indispensable in the implementation of deep learning models at scale. Such packages often have grounding in *functional programming*: perhaps JAX is the most direct example, as it is marketed as “*composable function transformations*”, but such features permeate other deep learning frameworks as well. Treating neural networks as “pure functions” allows for rigorous analysis on their computational graph, allowing a degree of type- and shape-checking, as well as automatic tensor shape inference and fully automated backpropagation passes.

The issues, again, happen closer to the boundary between the two directions—specifying and controlling for *constraint satisfaction* is not simple with tensor programming. Inferring general properties (*semantics*) of a program from its implementation (*syntax*) alone is a substantial challenge for all but the simplest programs, pointing to a need to model more abstract properties of **computer science** than existing frameworks can offer directly. The similarity of the requirement on both sides leads us to our present position.

## 1.2. Our position

It is our **position** that constructing a guiding framework for all of deep learning, requires robustly *bridging* the top-down and bottom-up approaches to neural network specification with a *unifying mathematical theory*, and that the concepts for this bridging should be coming from **computer science**. *Moreover, such a framework must generalise both group theory and functional programming—and a natural candidate for achieving this is category theory.*

To defend our position, we will demonstrate a unified categorical framework that is expressive enough to red-erive standard GDL concepts (invariance and equivariance), specify implementations of complex neural network build-

ing blocks (recurrent neural networks), as well as model other intricate deep learning concepts such as weight sharing.

## 1.3. The power of Category Theory

To understand where we are going, we must first put the field of category theory in context. Minimally, it may be conceived of as a battle-tested system of *interfaces* that are learned once, and then reliably applied across scientific fields. Originating in abstract mathematics, specifically algebraic topology, category theory has since proliferated, and been used to express ideas from numerous fields in an uniform manner, helping reveal their previously unknown shared aspects. Other than modern pure mathematics, which it thoroughly permeates, these fields include *systems theory* (Capucci et al., 2022; Niu & Spivak, 2023), *bayesian learning* (Braithwaite et al., 2023; Cho & Jacobs, 2019), and *information theory and probability* (Leinster, 2021; Bradley, 2021; Sturtz, 2015; Heunen et al., 2017; Perrone, 2022).

This growth has resulted in a reliable set of mature theories and tools; from algebra, geometry, topology, combinatorics to recursion and dependent types, etc. all of them with a mutually compatible interface. Recently category theory has started to be applied to machine learning, in *automatic differentiation* (Vákár & Smeding, 2022; Alvarez-Picallo et al., 2021; Gavranović, 2022), *topological data analysis* (Guss & Salakhutdinov, 2018), *natural language processing* (Lewis, 2019), *causal inference* (Jacobs et al., 2019), and even an entire categorical picture of gradient-based learning, from architectures to backprop (Cruttwell et al., 2022).

### 1.3.1. ESSENTIAL CONCEPTS

Before we begin, we recall three essential concepts in category theory, that will be necessary for following our exposition. First, we define a *category*, an elegant axiomatisation of a compositional structure.

**Definition 1.1** (Category). A category,  $\mathcal{C}$ , consists of a collection<sup>1</sup> of *objects*, and a collection of *morphisms* between pairs of objects, such that:

- For each object  $A \in \mathcal{C}$ , there is a unique *identity* morphism  $\text{id}_A : A \rightarrow A$ .
- For any two morphisms  $f : A \rightarrow B$  and  $g : B \rightarrow C$ , there must exist a unique morphism which is their *composition*  $g \circ f : A \rightarrow C$ .

subject to the following conditions:

<sup>1</sup>The term “collection”, rather than set, avoids Russell’s paradox, as objects may themselves be sets. Categories that can be described with sets are known as *small categories*.

- For any morphism  $f : A \rightarrow B$ , it holds that  $\text{id}_B \circ f = f \circ \text{id}_A = f$ .
- For any three composable morphisms  $f : A \rightarrow B$ ,  $g : B \rightarrow C$ ,  $h : C \rightarrow D$ , composition is *associative*, i.e.,  $h \circ (g \circ f) = (h \circ g) \circ f$ .

We denote by  $\mathcal{C}(A, B)$  the collection of all morphisms from  $A \in \mathcal{C}$  to  $B \in \mathcal{C}$ .

We provide a typical first example:

**Example 1.2** (The Set Category). Set is a category whose objects are *sets*, and morphisms are *functions* between them.

And another example, important for geometric DL:

**Example 1.3** (Groups and monoids as categories). A group,  $G$ , can be represented as a category,  $\mathcal{B}G$ , with a single object ( $G$ ), and morphisms  $g : G \rightarrow G$  corresponding to elements  $g \in G$ , where composition is given by the group's binary operation. Note that  $G$  is a group if and only if these morphisms are *isomorphisms*, that is, for each  $g : G \rightarrow G$  there exists a  $h : G \rightarrow G$  such that  $h \circ g = g \circ h = \text{id}_G$ . More generally, we can identify one-object categories, whose morphisms are not necessarily invertible, with monoids.

The power of category theory starts to emerge when we allow different categories to *interact*. Just as there are functions of sets and homomorphisms of groups, there is a more generic concept of *structure preserving maps* between categories, called *functors*.

**Definition 1.4** (Functor). Let  $\mathcal{C}$  and  $\mathcal{D}$  be two categories. Then,  $F : \mathcal{C} \rightarrow \mathcal{D}$  is a functor between them, if it maps each object and morphism of  $\mathcal{C}$  to a corresponding one in  $\mathcal{D}$ , and the following two conditions hold:

- For any object  $A \in \mathcal{C}$ ,  $F(\text{id}_A) = \text{id}_{F(A)}$ .
- For any composable morphisms  $f, g$  in  $\mathcal{C}$ ,  $F(g \circ f) = F(g) \circ F(f)$ .

An *endofunctor* on  $\mathcal{C}$  is a functor  $F : \mathcal{C} \rightarrow \mathcal{C}$ .

Just as a functor is an interaction between categories, a *natural transformation* specifies an interaction between functors; this is the third and final concept we cover here.

**Definition 1.5** (Natural transformation). Let  $F : \mathcal{C} \rightarrow \mathcal{D}$  and  $G : \mathcal{C} \rightarrow \mathcal{D}$  be two functors between categories  $\mathcal{C}$  and  $\mathcal{D}$ . A natural transformation  $\alpha : F \Rightarrow G$  consists of a choice, for every object  $X \in \mathcal{C}$ , of a morphism  $\alpha_X : F(X) \rightarrow G(X)$  in  $\mathcal{D}$  such that, for every morphism  $f : X \rightarrow Y$  in  $\mathcal{C}$ , it holds that  $\alpha_Y \circ F(f) = G(f) \circ \alpha_X$ .

The morphism  $\alpha_X$  is called the component of the natural transformation  $\alpha$  at the object  $X$ .

The components of a natural transformation assemble into “naturality squares”, commutative diagrams:

$$\begin{array}{ccc} F(X) & \xrightarrow{F(f)} & F(Y) \\ \alpha_X \downarrow & & \downarrow \alpha_Y \\ G(X) & \xrightarrow{G(f)} & G(Y) \end{array}$$

where a diagram *commutes* if, for any two objects, any two paths connecting them correspond to the same morphism.

## 2. From Monad Algebras to Equivariance

Having set up the essential concepts, we proceed on our quest to define a categorical framework which subsumes and generalises geometric deep learning (Bronstein et al., 2021). First, we will define a powerful notion (**monad algebra homomorphism**) and demonstrate that the special case of monads induced by *group actions* is sufficient to describe *geometric deep learning*. Generalising from monads and their algebras to arbitrary endofunctors and their algebras, we will find that our theory can express functions that process structured data from computer science (e.g. *lists* and *trees*) and behave in stateful ways like *automata*.

### 2.1. Monads and their algebras

**Definition 2.1** (Monad). Let  $\mathcal{C}$  be a category. A monad on  $\mathcal{C}$  is a triple  $(M, \eta, \mu)$  where  $M : \mathcal{C} \rightarrow \mathcal{C}$  is an endofunctor, and  $\eta : \text{id}_{\mathcal{C}} \Rightarrow M$  and  $\mu : M \circ M \Rightarrow M$  are natural transformations (where here  $\circ$  is functor composition), making diagrams in Definition B.1 commute.

**Example 2.2** (Group action monad). Let  $G$  be a group. Then the triple  $(G \times -, \eta, \mu)$  is a monad on  $\text{Set}$ , where

- $G \times - : \text{Set} \rightarrow \text{Set}$  is an endofunctor mapping a set  $X$  to the set  $G \times X$ ;
- $\eta : \text{id}_{\text{Set}} \Rightarrow G \times - : \text{Set} \rightarrow \text{Set}$  whose component at a set  $X$  is the function  $x \mapsto (e, x)$  where  $e$  is the identity element of the group  $G$ ; and
- $\mu : G \times G \times - \Rightarrow G \times - : \text{Set} \rightarrow \text{Set}$  whose component at a set  $X$  is the function  $(g, h, x) \mapsto (gh, x)$  with the implicit multiplication that of the group  $G$ .

Group action monads are formal theories of group actions, but they do not allow us to actually *execute* them on data. This is what *algebras* do.

**Definition 2.3** (Algebra for a monad). An algebra for a monad  $(M, \eta, \mu)$  on a category  $\mathcal{C}$  is a pair  $(A, a)$ , where  $A \in \mathcal{C}$  is a *carrier object* and  $a : M(A) \rightarrow A$  is a morphism of  $\mathcal{C}$  (*structure map*) making the following diagram

commute:

$$\begin{array}{ccc}
 A & \xrightarrow{\eta_A} & M(A) \\
 \searrow & & \downarrow a \\
 & & A
 \end{array}
 \qquad
 \begin{array}{ccc}
 M(M(A)) & \xrightarrow{M(a)} & M(A) \\
 \mu_A \downarrow & & \downarrow a \\
 M(A) & \xrightarrow{a} & A
 \end{array}$$

**Example 2.4** (Group actions). Group actions for a group  $G$  arise as algebras of the aforementioned group action monad  $G \times -$ . Consider the carrier  $\mathbb{R}^{\mathbb{Z}_w \times \mathbb{Z}_h}$ , thought of as data on a  $w \times h$  grid, and any of the usual group actions on  $\mathbb{Z}_w \times \mathbb{Z}_h$ : translation, rotation, permutation, scaling, or reflections.

Each of these group actions induce an algebra on the carrier set  $\mathbb{R}^{\mathbb{Z}_w \times \mathbb{Z}_h}$ . For instance, the translation group  $(\mathbb{Z}_w \times \mathbb{Z}_h, +, 0)$  induces the algebra

$$\blacktriangleright : \mathbb{Z}_w \times \mathbb{Z}_h \times \mathbb{R}^{\mathbb{Z}_w \times \mathbb{Z}_h} \rightarrow \mathbb{R}^{\mathbb{Z}_w \times \mathbb{Z}_h}$$

defined as  $((i', j') \blacktriangleright x)(i, j) = x(i - i', j - j')$ . Here  $x$  represents the grid data,  $i, j$  specific pixel locations, and  $i', j'$  the translation vector. We also specifically mention the trivial action of any group  $\pi_X : G \times X \rightarrow X$  by projection.

A monad algebra can capture a particular input or output for group equivariant neural networks (as its carrier). That being said, geometric deep learning concerns itself with *linear equivariant layers* between these inputs and outputs. In order to be able to describe those, we need to establish the concept of a *morphism of algebras* for a monad.

**Definition 2.5** ( $M$ -algebra homomorphism). Let  $(M, \mu, \eta)$  be a monad on  $\mathcal{C}$ , and  $(A, a)$  and  $(B, b)$  be  $M$ -algebras. An  $M$ -**algebra homomorphism**  $(A, a) \rightarrow (B, b)$  is a morphism  $f : A \rightarrow B$  of  $\mathcal{C}$  s.t. the following commutes:

$$\begin{array}{ccc}
 M(A) & \xrightarrow{M(f)} & M(B) \\
 a \downarrow & & \downarrow b \\
 A & \xrightarrow{f} & B
 \end{array}$$

We recover equivariant maps as morphisms of algebras.

**Example 2.6** (Equivariant maps). Equivariant maps are group action monad algebra homomorphisms. Consider any action from Example 2.4. An endomorphism of such an action—that is, a  $G$ -algebra on  $\mathbb{R}^{\mathbb{Z}_w \times \mathbb{Z}_h}$ —is an endomorphism of  $\mathbb{R}^{\mathbb{Z}_w \times \mathbb{Z}_h}$  which induces a commutative diagram

$$\begin{array}{ccc}
 G \times \mathbb{R}^{\mathbb{Z}_w \times \mathbb{Z}_h} & \xrightarrow{G \times f} & G \times \mathbb{R}^{\mathbb{Z}_w \times \mathbb{Z}_h} \\
 \blacktriangleright \downarrow & & \downarrow \blacktriangleright \\
 \mathbb{R}^{\mathbb{Z}_w \times \mathbb{Z}_h} & \xrightarrow{f} & \mathbb{R}^{\mathbb{Z}_w \times \mathbb{Z}_h}
 \end{array}$$

which, elementwise, unpacks to the equation

$$f(g \blacktriangleright x) = g \blacktriangleright f(x)$$

The translation example, for instance, recovers the equation  $f(((i', j') \blacktriangleright x)(i, j)) = (i', j') \blacktriangleright f(x)(i, j)$  which reduces to the usual constraint:  $f(x(i - i', j - j')) = f(x)(i - i', j - j')$ .

The concept of *invariance* — a special case of equivariance — is unpacked in Appendix (Example I.3). It's worth reflecting on the fact that we have just successfully derived the key aim of **geometric deep learning**: finding neural network layers that are *monad algebra homomorphisms* of monads associated with *group actions*!

To concretely derive such layers from these constraints, we need to make concrete the category in which  $f : A \rightarrow B$  lives. A standard choice is to use **Vect**, a category where objects are finite-dimensional *vector spaces* and morphisms are *linear maps* between these spaces. In such a setting, morphisms can be specified as *matrices*, and the equivariance condition places constraints on the matrix's entries, resulting in effects such as *weight sharing* or *weight tying*. We provide a detailed derivation for two examples on a two-pixel grid in Appendix I.

*Remark 2.7.* When our monad is of the form  $M \times -$ , with  $M$  a monoid, algebras are equivalent to  $M$ -actions, i.e. functors  $\mathcal{B}M \rightarrow \mathbf{Set}$ , where  $\mathcal{B}M$  is the one-object category given in Example 1.3, and algebra morphisms are equivalent to natural transformations. So in this case, our definition of equivariance coincides with the functorial version given in de Haan et al. (2020a). But the connection here is much deeper—for any monad, we can think of its algebras, which we can think of as the semantics of the monad, as functors on certain categories encoding the monad's syntax, such as *Lawvere theories*. For example, Dudzik & Veličković (2022) use the fact that functors on the category of finite polynomial diagrams<sup>2</sup> encode the algebraic structure of commutative semirings. We give further details on this connection in Appendix C.

## 2.2. Endofunctors and their (co)algebras

Geometric deep learning, while an elegant approach, is fundamentally constrained by the axioms of group theory. Monads and their algebras, however, are naturally generalised beyond group actions. Here we show how, by studying (co)algebras of arbitrary *endofunctors*, we can rediscover standard computer science constructs like trees, automata, and various algebraic notions. Then, in the next section we'll show how these more minimal structures, endofunctors and their algebras, may be augmented into the more structured notions of monads and their algebras.

**Definition 2.8** (Algebra for an endofunctor). Let  $\mathcal{C}$  be a category and  $F : \mathcal{C} \rightarrow \mathcal{C}$  an endofunctor on  $\mathcal{C}$ . An algebra

<sup>2</sup>Equivalently, the category of finitary dependent polynomial functors.



for  $F$  is a pair  $(A, a)$  where  $A$  is an object of  $\mathcal{C}$  and  $a : F(A) \rightarrow A$  is a morphism of  $\mathcal{C}$ .

Note that, compared to Definition 2.3, there are no equations this time;  $F$  is not equipped with any extra structure with which the structure map of an algebra could be compatible. Examples of endofunctor algebras abound (Jacobs, 2016), many of which are familiar to computer scientists.

**Example 2.9** (Lists). Let  $A$  be a set, and consider the endofunctor  $1 + A \times - : \text{Set} \rightarrow \text{Set}$ . The set  $\text{List}(A)$  of lists of elements of type  $A$  together with the map  $[\text{Nil}, \text{Cons}] : 1 + A \times \text{List}(A) \rightarrow \text{List}(A)$  forms an algebra of this endofunctor.<sup>3</sup> Here  $\text{Nil}$  and  $\text{Cons}$  are two constructors for lists, allowing us to represent lists as the following datatype:

---

```
data List a = Nil
            | Cons a (List a)
```

---

It describes  $\text{List}(A)$  *inductively*, as being formed either out of the empty list, or an element of type  $A$  and another list. In Figure 1 we will see how this relates to *folding* RNNs.

**Example 2.10** (Binary trees). Let  $A$  be a set. Consider the endofunctor  $A + (-)^2 : \text{Set} \rightarrow \text{Set}$ . The set  $\text{Tree}(A)$  of binary trees with  $A$ -labelled leaves, together with the map  $[\text{Leaf}, \text{Node}] : A + \text{Tree}(A)^2 \rightarrow \text{Tree}(A)$  forms an algebra of this endofunctor. Here  $\text{Leaf}$  and  $\text{Node}$  are constructors for binary trees, enabling the following datatype representation:

---

```
data Tree a = Leaf a
            | Node (Tree a) (Tree a)
```

---

It describes  $\text{Tree}(A)$  *inductively*, as being formed either out of a single  $A$ -labelled leaf or two subtrees.<sup>4</sup> In Figure 1 we will relate this to recursive neural networks.

Dually, we also study *coalgebras* for an endofunctor (where the structure morphism  $a : A \rightarrow F(A)$  points the other way (Definition B.2). Intuitively, while algebras offer us a way to model computation guaranteed to terminate, coalgebras offer us a way to model potentially infinite computation. They capture the semantics of programs whose guarantee is not termination, but rather *productivity* (Atkey & McBride, 2013), and as such are excellent for describing servers, operating systems, and automata (Rutten, 2000; Jacobs, 2016). We will use endofunctor coalgebras to describe one such automaton—the Mealy machine (Mealy, 1955).

**Example 2.11** (Mealy machines). Let  $O$  and  $I$  be sets

<sup>3</sup> $[f, g] : A + B \rightarrow C$  is notation for maps out of a coproduct; where  $f : A \rightarrow C$  and  $g : B \rightarrow C$

<sup>4</sup>This framework can also model any variations, e.g.,  $n$ -ary trees with  $A$ -labelled leaves as algebras of  $A + \text{List}(-)$ , or binary trees with  $A$ -labelled nodes as algebras of  $1 + A \times (-)^2$ .

of possible *outputs* and *inputs*, respectively. Consider the endofunctor  $(I \rightarrow O \times -) : \text{Set} \rightarrow \text{Set}$ . Then the set  $\text{Mealy}_{O,I}$  of Mealy machines with outputs in  $O$  and inputs in  $I$ , together with the map  $\text{next} : \text{Mealy}_{O,I} \rightarrow (I \rightarrow O \times \text{Mealy}_{O,I})$  is a coalgebra of this endofunctor.

---

```
data Mealy o i = MkMealy {
    next :: i -> (o, Mealy o i)
}
```

---

This describes Mealy machines *coinductively*, as *systems* which, given an input, produce an output and another Mealy machine. In Figure 1 we will relate this to full recurrent neural networks, and, in Examples I.4 and I.7 we coalgebraically express two other fundamental classes of automata: streams and Moore machines.

We have expressed data structures and automata using (co)algebras for an endofunctor. Just as in the case of GDL, in order to describe (linear) *layers of neural networks* between them, we need to establish the concept of a *homomorphism of endofunctor (co)algebras*. The definition of a homomorphism of algebras for an endofunctor mirrors<sup>5</sup> Definition 2.5, while the definition of a homomorphism of coalgebras has the structure maps pointing the other way.

**Example 2.12** (Folds over lists as algebra homomorphisms). Consider the endofunctor  $(1 + A \times -)$  from Example 2.9, and an algebra homomorphism from  $(\text{List}(A), [\text{Nil}, \text{Cons}])$  to any other  $(1 + A \times -)$ -algebra  $(X, [r_0, r_1])$ :

$$\begin{array}{ccc} 1 + A \times \text{List}(A) & \xrightarrow{1 + A \times f_r} & 1 + A \times X \\ \downarrow [\text{Nil}, \text{Cons}] & & \downarrow [r_0, r_1] \\ \text{List}(A) & \xrightarrow{f_r} & X \end{array}$$

Then the map  $f_r : \text{List}(A) \rightarrow X$  is, necessarily, a *fold* over a list, a concept from functional programming which describes how a single value is obtained by operating over a list of values. It is implemented by recursion on the input:

---

```
f_r :: List a -> x
f_r Nil = r_0 ()
f_r (Cons h t) = r_1 h (f_r t)
```

---

This recursion is *structural* in nature, meaning it satisfies the following two equations which arise by unpacking the algebra homomorphism equations elementwise:

$$f_r(\text{Nil}) = r_0(\bullet) \quad (1)$$

$$f_r(\text{Cons}(h, t)) = r_1(h, f_r(t)) \quad (2)$$

Equation (1) tells us that we get the same result if we apply  $f_r$  to  $\text{Nil}$  or apply  $r_0$  to the unique element of the singleton

<sup>5</sup>Because it does not rely on the extra structure monads have.

set<sup>6</sup>. Equation (2) tells us that, starting with the head and tail of a list, we get the same result if we concatenate the head to the tail, and then process the entire list with  $f_r$ , or if we process the tail first with  $f_r$ , and then combine the result with the head using  $r_1$ .

It is important to remark that these equations **generalise equivariance constraints** over a list structure. Both group equivariance and Equations (1) and (2) intuitively specify a function that is predictably affected by certain operations—but for the case of lists, these operations (concatenating) are *not* group actions, as attaching an element to the front of the list does *not* leave the list unchanged.

**Remark 2.13.** Interestingly, given an algebra  $(X, [r_0, r_1])$ , there can only ever be *one* algebra homomorphism from lists to it! This is because  $(\text{List}(A), [\text{Nil}, \text{Cons}])$  is an *initial object* (Definition A.2) in the category of  $(1 + A \times -)$ -algebras. The fact that these are initial arises from a deeper fact related to the fact that, in many cases, for a given endofunctor there is a monad whose category of monad algebras is equivalent to the original category of endofunctor algebras. We note this because the construction which takes us from one to the other, the so-called *algebraically free monad* on an endofunctor, will be seen in Part 3 to derive RNNs and other similar architectures from first principles.

**Example 2.14** (Tree folds as algebra homomorphisms). Consider the endofunctor  $A + (-)^2$  from Example 2.10, and an algebra homomorphism from  $(\text{Tree}(A), [\text{Leaf}, \text{Node}])$  to any other  $(A + (-)^2)$ -algebra  $(X, [r_0, r_1])$ :

$$\begin{array}{ccc} A + \text{Tree}(A)^2 & \xrightarrow{A + f_r^2} & A + X^2 \\ \downarrow [\text{Leaf}, \text{Node}] & & \downarrow [r_0, r_1] \\ \text{Tree}(A)^2 & \xrightarrow{f_r} & X \end{array}$$

Then the map  $f_r$  is necessarily a *fold* over a tree. As with lists, it is implemented by recursion on the input, which is *structural* in nature:

---

```
f_r :: Tree a -> x
f_r (Leaf a) = r_0 a
f_r (Node l r) = r_1 (f_r l) (f_r r)
```

---

This means that it satisfies the following two equations which arise by unpacking the algebra homomorphism equations elementwise:

$$f_r(\text{Leaf}(a)) = r_0(a) \quad (3)$$

$$f_r(\text{Node}(l, r)) = r_1(f_r(l), f_r(r)) \quad (4)$$

These can also be thought as describing generalised equivariance over binary trees, analogously to lists.

<sup>6</sup>Where  $()$  was used to denote it in Haskell notation.

Dual to algebra homomorphisms and folds over inductive data structures, *coalgebra homomorphisms* are categorical semantics of *unfolds* over coinductive data structures.

**Example 2.15** (Unfolds as coalgebra homomorphisms). Consider the endofunctor  $(I \rightarrow O \times -)$  from Example 2.11, and a coalgebra homomorphism from  $(\text{Mealy}_{O,I}, \text{next})$  to any other  $(I \rightarrow O \times -)$ -coalgebra  $(X, n)$ :

$$\begin{array}{ccc} X & \xrightarrow{f_n} & \text{Mealy}_{O,I} \\ n \downarrow & & \downarrow \text{next} \\ (I \rightarrow O \times X) & \xrightarrow{(I \rightarrow O \times f_n)} & (I \rightarrow O \times \text{Mealy}_{O,I}) \end{array}$$

The map  $f_n$  here can be thought of as a generalised *unfold*, a concept from functional programming describing how a potentially infinite data structure is obtained from a single value. It is implemented by a *corecursive* function:

---

```
f_n :: x -> Mealy o i
f_n x = MkMealy
  \i -> let (o', x') = n x i
        in (o', f_n x')
```

---

which is again structural in nature. This means that it satisfies the following two equations which arise by unpacking the coalgebra homomorphism equations elementwise:

$$n(x)(i)_1 = \text{next}(f_n(x))(i)_1 \quad (5)$$

$$f_n(n(x)(i)_2) = \text{next}(f_n(x))(i)_2 \quad (6)$$

Equation (5) tells us that the output of the Mealy machine produced by  $f_n$  at state  $x$  and input  $i$  is given by the output of  $n$  at state  $x$  and input  $i$ , and Equation (6) tells us that the next Mealy machine produced at  $x$  and  $i$  is the one produced by  $f_n$  at  $n(x)_2(i)$ .

This, too, **generalises equivariance** constraints, now describing an interactive automaton which is by no means invertible. Instead, it is *dynamic* in nature, producing outputs which are dependent on the current state of the machine and previously unknown inputs. Lastly, in Examples I.5 and I.8, we show that two kinds of automata—streams and Moore machines—are also examples of coalgebra homomorphisms. Just as before, we can embed all of our objects and morphisms into Vect to study the weight sharing constraints induced by such a condition—see Example I.5.

### 2.3. Where to next?

Let's take a step back and understand what we've done. We have shown that an existing categorical framework *uniformly* captures a number of different data structures and automata, as particular (co)algebras of an endofunctor. By choosing a well-understood data structure, we induce a

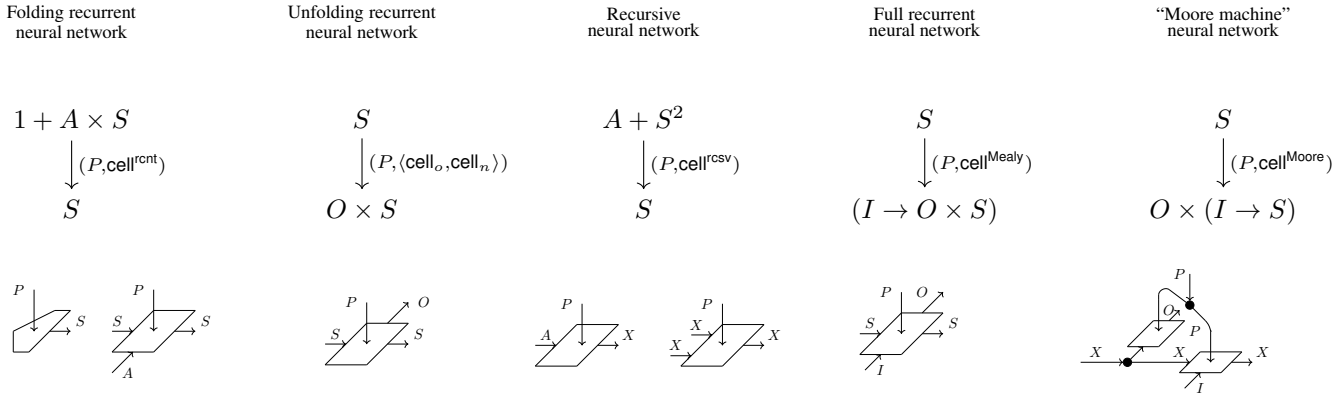


Figure 1. Parametric (co)algebras provide a high-level framework for describing structured computation in neural networks.

structural constraint on the control flow of the corresponding neural network, by utilising homomorphisms of these endofunctor (co)algebras. These follow the same recipe as monad algebra homomorphisms, and hence can be thought of as *generalising equivariance*—describing functions *beyond* what geometric deep learning can offer.

This is concrete evidence for our position—that **categorical algebra homomorphisms** are suitable for capturing various constraints one can place on deep learning architectures. Our evidence so far rested on *endofunctor algebras*, which are a particularly fruitful variant.

However, this construct leaves much to be desired. One major issue is that, to prescribe any notion of *weight sharing*, for all of these examples we have implicitly assumed homomorphisms to be linear transformations by placing them into the category Vect. But most neural networks aren’t simply linear maps, meaning that these analyses are limited to analysing their individual layers. In the standard example of recurrent neural networks, an RNN cell is an *arbitrary* differentiable function, usually composed out of a sequence of linear and non-linear maps. Further, in both practice and theory, neural networks are treated as *parametric* functions.

How can we explicitly model parameters and non-linear maps, without abandoning the presented categorical framework? Furthermore, in practice—with recurrent or recursive (Socher et al., 2013) neural networks, for instance—there are established techniques for weight sharing. Can we establish formal criteria for when these techniques are *correct*? Just as how we generalised GDL to the setting of category theory, we can go further: to the setting of *2-categories*—the setting we use to study *parametric morphisms*.

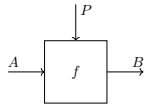
## 3. 2-Categories and Parametric Morphisms

While category theory is a powerful framework, it leaves much to be desired in terms of *higher-order relationships* between morphisms. It only deals with *sets* of morphisms, with no possible way to *compare* elements of these sets. This is where the theory of *2-categories* comes in, which deals with an entire *category* of morphisms. While in a (1-)category, one has objects and morphisms between objects, in a 2-category one has objects (known as *0-morphisms*), morphisms between objects (*1-morphisms*), and morphisms between *morphisms* (*2-morphisms*). We have, in fact, already secretly seen an instance of a 2-category, Cat, when defining the essential concepts of category theory. Specifically, in Cat, objects are *categories*, morphisms are *functors* between them, and 2-morphisms are *natural transformations* between functors.

### 3.1. The 2-category Para

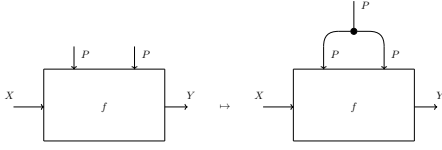
Here we study an established 2-category, called Para (Cruttwell et al., 2022; Capucci et al., 2022). While it shares objects with the category Set, its 1-morphisms are not functions, but *parametric* functions. That is, a 1-morphism  $A \rightarrow B$  here consists of a pair  $(P, f)$ , where  $P \in \text{Set}$  and  $f : P \times A \rightarrow B$ .

Para morphisms admit an elegant graphical formalism. Parameters ( $P$ ) are drawn *vertically*, signifying that they are part of the *morphism*, and not objects.



The 2-category Para models the algebra of composition of neural networks; the sequential composition of parametric morphisms composes the parameter spaces in parallel (Figure 4).

The 2-morphisms in  $\text{Para}$  capture *reparameterisations* between parametric functions. Importantly, this allows for the **explicit treatment of weight tying**, where a parametric morphism  $(P \times P, f)$  can have its weights tied by precomposing with the copy map  $\Delta_P : P \rightarrow P \times P$ .



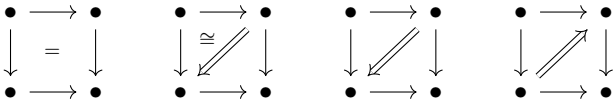
This 2-category<sup>7</sup> is one of the key constructions in the categorical picture of full gradient-based learning (Cruttwell et al., 2022). But more is true:

The 2-category  $\text{Para}$  and 2-categorical algebra valued in it provide a **formal theory of neural network architectures**, establish **formal criteria** for correctness of weight tying and **inform design** of new architectures.

### 3.2. 2-dimensional categorical algebra

2-category theory is markedly richer than 1-category theory.

While diagrams in a 1-category either commute or do not commute, in a 2-category, they serve as a *1-skeleton* to which 2-morphisms *attach*. In any 2-category a square may: commute, pseudo-commute, lax-commute, or oplax<sup>8</sup> commute, meaning, respectively, that relevant paths are equal, isomorphic, or there is a 2-morphism from one to the other in one direction or the other. The diagrams below present these four options, with 2-morphisms denoted by double arrows.



In the long run, we expect that *all* of these notions will apply to, either explaining or specifying, aspects of neural architecture past, present and future. Focusing on just one of them, the *lax algebras* are sufficient to derive *recursive*, *recurrent*, and similar neural networks from first principles. Notably, morphisms of lax algebras are also expressive enough to capture *1-cocycles*, used to formalise asynchronous neural networks in (Dudzik et al., 2023)—see Appendix I.

Interestingly, this story of how an individual recurrent, recursive, etc. neural network cell generates a full recursive, recurrent etc. neural network is a particular 2-categorical

analogue to the story of *algebraically free monads* on an endofunctor we briefly mentioned in Remark 2.13.

For all the examples of endofunctors in Section 2.2, there is a monad whose category of algebras  $\text{Free}_{\text{Mnd}}(F)$  is equivalent to the category of algebras for the original endofunctor  $F$ . We obtain  $\text{Free}_{\text{Mnd}}(F)$  by iterating  $F$  until it *stabilises*, meaning further application of the endofunctor does not change the composition. Functional programmers may recognise this from the implementation of free monads in Haskell, while formally this is defined using *colimits* (see Appendix B.2). Using this concept, we can define a functor mapping an  $F$ -algebra  $(A, a)$  to the  $\text{Free}_{\text{Mnd}}(F)$ -algebra  $(A, \lim(a \circ F a \circ F^2 a \circ \dots \circ F^n a))$ , connecting appropriate endofunctor algebras to monad algebras.

But in the 2-dimensional case, we study the relationship between  $\text{Lax-Alg}_{\text{Endo}}(F)$  and  $\text{Lax-Alg}_{\text{Mnd}}(\text{Free}_{\text{Mnd}}(F))$  and need contend not only with generating the 1-dimensional structure map, but also the 2-cells of the lax algebra for a monad.

To reconcile this with concrete applications, we note that we do not need to study general 2-endofunctors and 2-monads on  $\text{Para}$ . Rather, examples which concern us arise from specific 1-categorical algebras (group action monads, inductive types, etc.), which are *augmented* into 2-monads on  $\text{Para}$ . As we prove in Theorem G.10, the lax cells of such algebras are actually *comonoids*. The fact that we can duplicate or delete entries in vectors—the essence of tying weights—is the informal face of this comonoid structure.

We can now describe, even if space constraints prevent us from adequate level of detail, the universal properties of recurrent, recursive, and similar models: they are *lax algebras* for *free parametric monads* generated by *parametric endofunctors*! Having lifted the concept of algebra introduced in Part 2 into 2-categories, we can now describe several influential neural networks *fully* (not just their individual layers!) from first principles of functional programming.

## 4. New Horizons

Our framework gives the *correct definition* of numerous variants of structured networks as universal parametric counterparts of known notions in computer science. This immediately opens up innumerable avenues for research.

Firstly, we have seen that monad algebras—which *generalise* equivariance constraints—can be parametric, and *lax*. As a consequence, the kinds of equivariance constraints we can *learn* become more general: we hypothesise neural networks that can learn not merely conservation laws (as in Alet et al. (2021)), but verifiably correct logical argument, or code. This has ramifications for *code synthesis*: we can,

<sup>7</sup>More precisely, the construction  $\text{Para}(\_)$ . See Appendix G.

<sup>8</sup>Often also called *colax*.



for example, specify neural networks that learn only *well-typed functions* by choosing appropriate algebras as their domain and codomain.

This is made possible by our framework’s generality: by choosing *polynomial functors* as endofunctors we get access to *containers* (Abbott et al., 2003; Altenkirch et al., 2010), a uniform way to program with and reason about datatypes and polymorphic functions. By combining these insights with recent advances enabling *purely functional* differentiation through inductive and coinductive types (Nunes & Vákár, 2023), we open new vistas for *type-safe* design and implementation of neural networks in functional languages.

Our framework offers a *proactive* path towards *equitable* AI systems. GDL already enables the construction of falsifiable tests for equivariance and invariance—e.g., to protected classes such as race and gender—enabling well-defined fairness, which is already an improvement on techniques requiring properties of data sets to which real-world data explicitly does not conform. Our framework additionally allows us to specify the kinds of *arguments* the neural networks can use to come to their conclusions. This is a level of expressivity permitting reliable use for assessing bias, fairness in *the kinds of reasoning* in AI models deployed at scale. We thus believe that this is the right path to AI compliance and safety, and not merely explainable, but verifiable AI.

## References

- Abadi, M., Agarwal, A., Barham, P., Brevdo, E., Chen, Z., Citro, C., Corrado, G. S., Davis, A., Dean, J., Devin, M., et al. Tensorflow: Large-scale machine learning on heterogeneous distributed systems. *arXiv preprint arXiv:1603.04467*, 2016.
- Abbott, M., Altenkirch, T., and Ghani, N. Categories of containers. In Gordon, A. D. (ed.), *Foundations of Software Science and Computation Structures*, Lecture Notes in Computer Science, pp. 23–38. Springer, 2003. ISBN 978-3-540-36576-1. doi: 10.1007/3-540-36576-1\_2.
- Alet, F., Doblar, D., Zhou, A., Tenenbaum, J., Kawaguchi, K., and Finn, C. Noether networks: meta-learning useful conserved quantities. *Advances in Neural Information Processing Systems*, 34:16384–16397, 2021.
- Altenkirch, T., Levy, P., and Staton, S. Higher-order containers. In Ferreira, F., Löwe, B., Mayordomo, E., and Mendes Gomes, L. (eds.), *Programs, Proofs, Processes*, volume 6158, pp. 11–20. Springer Berlin Heidelberg, 2010. ISBN 978-3-642-13961-1 978-3-642-13962-8. doi: 10.1007/978-3-642-13962-8\_2. Series Title: Lecture Notes in Computer Science.
- Alvarez-Picallo, M., Ghica, D., Sprunger, D., and Zanasi, F. Functorial string diagrams for reverse-mode automatic differentiation, 2021.
- Atkey, R. and McBride, C. Productive coprogramming with guarded recursion. In *Proceedings of the 18th ACM SIGPLAN international conference on Functional programming*, ICFP ’13, pp. 197–208. Association for Computing Machinery, 2013. ISBN 978-1-4503-2326-0. doi: 10.1145/2500365.2500597. URL <https://doi.org/10.1145/2500365.2500597>.
- Bradbury, J., Frostig, R., Hawkins, P., Johnson, M. J., Leary, C., Maclaurin, D., Necula, G., Paszke, A., VanderPlas, J., Wanderman-Milne, S., et al. Jax: composable transformations of python+ numpy programs. 2018.
- Bradley, T.-D. Entropy as a topological operad derivation. 23(9):1195, 2021. ISSN 1099-4300. doi: 10.3390/e23091195. URL <http://arxiv.org/abs/2107.09581>.
- Braithwaite, D., Hedges, J., and St Clere Smithe, T. The compositional structure of bayesian inference. In *DROPS-IDN/v2/document/10.4230/LIPIcs.MFCS.2023.24*. Schloss-Dagstuhl - Leibniz Zentrum für Informatik, 2023. doi: 10.4230/LIPIcs.MFCS.2023.24.
- Brandenburg, M. Large limit sketches and topological space objects. *arXiv preprint arXiv:2106.11115*, 2021.
- Bronstein, M. M., Bruna, J., Cohen, T., and Veličković, P. Geometric deep learning: Grids, groups, graphs, geodesics, and gauges. *arXiv preprint arXiv:2104.13478*, 2021.
- Capucci, M. and Gavranović, B. Actegories for the working amthematician, 2023. URL <http://arxiv.org/abs/2203.16351>.
- Capucci, M., Gavranović, B., Hedges, J., and Rischel, E. F. Towards foundations of categorical cybernetics. 372:235–248, 2022. ISSN 2075-2180. doi: 10.4204/EPTCS.372.17. URL <http://arxiv.org/abs/2105.06332>.
- Chami, I., Abu-El-Haija, S., Perozzi, B., Ré, C., and Murphy, K. Machine learning on graphs: A model and comprehensive taxonomy. *The Journal of Machine Learning Research*, 23(1):3840–3903, 2022.
- Cho, K. and Jacobs, B. Disintegration and bayesian inversion via string diagrams. 29

- (7):938–971, 2019. ISSN 0960-1295, 1469-8072. doi: 10.1017/S0960129518000488. URL <http://arxiv.org/abs/1709.00322>.
- Cohen, T. S., Geiger, M., Köhler, J., and Welling, M. Spherical cnns. *arXiv preprint arXiv:1801.10130*, 2018.
- Cohen, T. S., Geiger, M., and Weiler, M. A general theory of equivariant cnns on homogeneous spaces. *Advances in neural information processing systems*, 32, 2019.
- Cruttwell, G. S. H., Gavranović, B., Ghani, N., Wilson, P., and Zanasi, F. Categorical foundations of gradient-based learning. In Sergey, I. (ed.), *Programming Languages and Systems*, Lecture Notes in Computer Science, pp. 1–28. Springer International Publishing, 2022. ISBN 978-3-030-99336-8. doi: 10.1007/978-3-030-99336-8\_1.
- de Haan, P., Cohen, T. S., and Welling, M. Natural graph networks. *Advances in neural information processing systems*, 33:3636–3646, 2020a.
- de Haan, P., Weiler, M., Cohen, T., and Welling, M. Gauge equivariant mesh cnns: Anisotropic convolutions on geometric graphs. *arXiv preprint arXiv:2003.05425*, 2020b.
- Dudzik, A., von Glehn, T., Pascanu, R., and Veličković, P. Asynchronous algorithmic alignment with cocycles. *arXiv preprint arXiv:2306.15632*, 2023.
- Dudzik, A. J. and Veličković, P. Graph neural networks are dynamic programmers. *Advances in Neural Information Processing Systems*, 35:20635–20647, 2022.
- Fey, M. and Lenssen, J. E. Fast graph representation learning with pytorch geometric. *arXiv preprint arXiv:1903.02428*, 2019.
- Fuchs, F., Worrall, D., Fischer, V., and Welling, M. Se (3)-transformers: 3d roto-translation equivariant attention networks. *Advances in neural information processing systems*, 33:1970–1981, 2020.
- Fukushima, K., Miyake, S., and Ito, T. Neocognitron: A neural network model for a mechanism of visual pattern recognition. *IEEE transactions on systems, man, and cybernetics*, (5):826–834, 1983.
- Gambino, N. and Kock, J. Polynomial functors and polynomial monads. In *Mathematical proceedings of the cambridge philosophical society*, volume 154, pp. 153–192. Cambridge University Press, 2013.
- Gavranović, B. Space-time tradeoffs of lenses and optics via higher category theory, 2022. URL <http://arxiv.org/abs/2209.09351>.
- Geiger, M. and Smidt, T. e3nn: Euclidean neural networks, 2022. URL <https://arxiv.org/abs/2207.09453>.
- Ghani, N., Lüth, C., de Marchi, F., and Power, J. Algebras, coalgebras, monads and comonads. 44(1):128–145. ISSN 1571-0661. doi: 10.1016/S1571-0661(04)80905-8.
- Gilmer, J., Schoenholz, S. S., Riley, P. F., Vinyals, O., and Dahl, G. E. Neural message passing for quantum chemistry. In *International conference on machine learning*, pp. 1263–1272. PMLR, 2017.
- Godwin, J., Keck, T., Battaglia, P., Bapst, V., Kipf, T., Li, Y., Stachenfeld, K., Veličković, P., and Sanchez-Gonzalez, A. Jraph: A library for graph neural networks in jax., 2020. URL <http://github.com/deepmind/jraph>.
- Guss, W. H. and Salakhutdinov, R. On characterizing the capacity of neural networks using algebraic topology, 2018. URL <http://arxiv.org/abs/1802.04443>.
- Heunen, C., Kammar, O., Staton, S., and Yang, H. A convenient category for higher-order probability theory. In *2017 32nd Annual ACM/IEEE Symposium on Logic in Computer Science (LICS)*, pp. 1–12, 2017. doi: 10.1109/LICS.2017.8005137. URL <http://arxiv.org/abs/1701.02547>.
- Jacobs, B. *Introduction to Coalgebra: Towards Mathematics of States and Observation*, volume 59 of *Cambridge Tracts in Theoretical Computer Science*. Cambridge University Press, 2016. ISBN 978-1-316-82318-7. doi: 10.1017/CBO9781316823187. URL <https://doi.org/10.1017/CBO9781316823187>.
- Jacobs, B., Kissinger, A., and Zanasi, F. Causal inference by string diagram surgery. In Bojańczyk, M. and Simpson, A. (eds.), *Foundations of Software Science and Computation Structures*, Lecture Notes in Computer Science, pp. 313–329. Springer International Publishing, 2019. ISBN 978-3-030-17127-8. doi: 10.1007/978-3-030-17127-8\_18.
- Jogl, F., Thiessen, M., and Gärtner, T. Expressivity-preserving gnn simulation. In *Thirty-seventh Conference on Neural Information Processing Systems*, 2023.
- Johnson, N., Yau, D., Johnson, N., and Yau, D. *2-Dimensional Categories*. Oxford University Press. ISBN 978-0-19-887137-8.
- Kelly, G. A unified treatment of transfinite constructions for free algebras, free monoids, colimits, associated sheaves, and so on. *Bulletin of the Australian Mathematical Society*, 22:1 – 83, 1980.

- Kelly, G. The basic concepts of enriched category theory. *Reprints in Theory and Applications of Categories [electronic only]*, 2005, 01 2005.
- Lack, S. *A 2-categories companion*, pp. 105–191. IMA volumes in mathematics and its applications. Springer, Springer Nature, United States, 2010. ISBN 9781441915238. doi: 10.1007/978-1-4419-1524-5.4.
- LeCun, Y., Bottou, L., Bengio, Y., and Haffner, P. Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, 86(11):2278–2324, 1998.
- Leinster, T. *Entropy and Diversity: The Axiomatic Approach*. Cambridge University Press, 2021. doi: 10.1017/9781108963558.
- Lewis, M. Compositionality for recursive neural networks. 6(4):709–724, 2019.
- Mealy, G. H. A method for synthesizing sequential circuits. 34(5):1045–1079, 1955. ISSN 0005-8580. doi: 10.1002/j.1538-7305.1955.tb03788.x. Conference Name: The Bell System Technical Journal.
- Niu, N. and Spivak, D. I. *Polynomial Functors: A Mathematical Theory of Interaction*. arXiv, 2023. URL <http://arxiv.org/abs/2312.00990>.
- Nunes, F. L. and Vákár, M. CHAD for expressive total languages. 33(4):311–426, 2023. ISSN 0960-1295, 1469-8072. doi: 10.1017/S096012952300018X. URL <http://arxiv.org/abs/2110.00446>.
- Papillon, M., Sanborn, S., Hajj, M., and Miolane, N. Architectures of topological deep learning: A survey on topological neural networks. *arXiv preprint arXiv:2304.10031*, 2023.
- Paszke, A., Gross, S., Massa, F., Lerer, A., Bradbury, J., Chanan, G., Killeen, T., Lin, Z., Gimelshein, N., Antiga, L., et al. Pytorch: An imperative style, high-performance deep learning library. *Advances in neural information processing systems*, 32, 2019.
- Perrone, P. Markov categories and entropy, 2022. URL <http://arxiv.org/abs/2212.11719>.
- Rutten, J. J. M. M. Universal coalgebra: a theory of systems. 249(1):3–80, 2000. ISSN 0304-3975. doi: 10.1016/S0304-3975(00)00056-6.
- Socher, R., Perelygin, A., Wu, J., Chuang, J., Manning, C. D., Ng, A., and Potts, C. Recursive deep models for semantic compositionality over a sentiment treebank. In *Proceedings of the 2013 Conference on Empirical Methods in Natural Language Processing*, pp. 1631–1642. Association for Computational Linguistics, 2013. URL <https://aclanthology.org/D13-1170>.
- Sturtz, K. Categorical probability theory, 2015. URL <http://arxiv.org/abs/1406.6030>.
- Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., and Polosukhin, I. Attention is all you need. *Advances in neural information processing systems*, 30, 2017.
- Vákár, M. and Smeding, T. CHAD: Combinatory homomorphic automatic differentiation. 44(3):20:1–20:49, 2022. ISSN 0164-0925. doi: 10.1145/3527634. URL <https://doi.org/10.1145/3527634>.
- Wang, M. Y. Deep graph library: Towards efficient and scalable deep learning on graphs. In *ICLR workshop on representation learning on graphs and manifolds*, 2019.
- Weiler, M., Forré, P., Verlinde, E., and Welling, M. *Equivariant and Coordinate Independent Convolutional Networks*. 2023.
- Worrall, D. and Welling, M. Deep scale-spaces: Equivariance over scale. *Advances in Neural Information Processing Systems*, 32, 2019.
- Xu, K., Li, J., Zhang, M., Du, S. S., Kawarabayashi, K.-i., and Jegelka, S. What can neural networks reason about? *arXiv preprint arXiv:1905.13211*, 2019.

## A. Category theory basics

**Proposition A.1.** Let  $X$  be a set, and let  $C$  be a vector space. Then given any group  $G$  and a group action  $\blacktriangleright: G \times X \rightarrow X$  the map  $F_{C-\text{Vect}}(\blacktriangleright)$  interpreted as the a map  $\coprod_G F_{C-\text{Vect}}(X) \rightarrow F_{C-\text{Vect}}(Y) \xrightarrow{\sim} G \times C^X \rightarrow C^Y$  is an action. Moreover it can be written  $(g \blacktriangleright x)(u) = x(g^{-1} \blacktriangleright u)$ , where we are treating  $C^X$  as a vector space whose structure arises pointwise from  $C$ .

*Proof.* Purely formal and left to the reader. □

**Definition A.2** (Initial object). An object  $I$  in a category  $\mathcal{C}$  is called **initial** if for every  $X : \mathcal{C}$  it naturally holds that  $\mathcal{C}(I, X) \cong 1$ , meaning that there is only one map of type  $I \rightarrow X$ .

**Definition A.3** (Terminal object). An object  $T$  in a category  $\mathcal{C}$  is called **terminal** if for every  $X : \mathcal{C}$  it naturally holds that  $\mathcal{C}(X, T) \cong 1$ , meaning that there is only one map of type  $X \rightarrow T$ .

## B. 1-Categorical algebra

**Definition B.1** (Monad coherence diagrams). A triple  $(M, \eta, \mu)$  of:

- an endofunctor  $M : \mathcal{C} \rightarrow \mathcal{C}$ ;
- a natural transformations  $\eta : \text{id}_{\mathcal{C}} \Rightarrow M$ ; and
- a natural transformation  $\mu : M \circ M \Rightarrow M$

constitute a **monad** if the following diagrams commute:

$$\begin{array}{ccc} M & \xrightarrow{\eta} & M \circ M \\ & \searrow & \downarrow \mu \\ & & M \end{array} \qquad \begin{array}{ccc} M \circ M \circ M & \xrightarrow{M \circ \mu} & M \circ M \\ \mu \downarrow & & \downarrow \mu \\ M \circ M & \xrightarrow{\mu} & M \end{array}$$

**Definition B.2** (Coalgebra for an endofunctor). Let  $\mathcal{C}$  be a category and  $F$  an endofunctor on  $\mathcal{C}$ . A coalgebra for  $F$  is a pair  $(A, a)$  where  $A$  is an object of  $\mathcal{C}$  and  $a : A \rightarrow F(A)$  is a morphism of  $\mathcal{C}$ .

### B.1. Well-pointed endofunctors and algebraically free monads

In this subsection we'll relate endofunctors and their algebras to monads and their algebras by way of *well-pointed endofunctors* and the transfinite construction for an *algebraically free monad* on such an endofunctor.

**Definition B.3.** A **pointed endofunctor**  $(F, \sigma)$  comprises an endofunctor  $F : \mathcal{C} \rightarrow \mathcal{C}$ , and a natural transformation  $\sigma : \text{id}_{\mathcal{C}} \Rightarrow F$ . A pointed endofunctor is said to be *well-pointed* if  $\sigma F = F \sigma : F \Rightarrow FF : \mathcal{C} \rightarrow \mathcal{C}$ .

An **algebra for a pointed endofunctor**  $(F, \sigma)$  is an algebra  $(A, a)$  for the endofunctor  $F$  for which the following diagram commutes:

$$\begin{array}{ccc} A & \xrightarrow{\sigma_A} & FA \\ & \searrow \text{id}_A & \downarrow a \\ & & A \end{array}$$

*Remark B.4.* Algebras for well-pointed endofunctors are just algebras for the pointed endo-functor. Well-pointedness is a property, not structure. Morphisms of algebras for pointed endofunctors are morphisms of algebras for the underlying endofunctor.

**Example B.5** (Monads subsume pointed endofunctors, idempotent monads subsume well-pointed endofunctors). All monads subsume pointed endofunctors. Idempotent monads subsume well-pointed endofunctors.

**Definition B.6.** For a given endofunctor  $F$ , its algebras and algebra homomorphisms form a category we denote by  $\text{Alg}_{\text{Endo}}(F)$ . For  $(F, \sigma)$  a (well)-pointed endofunctor we denote by  $\text{Alg}_{\text{Pendo}}(F)$  the category of algebras for  $F$  and homomorphisms thereof. For  $(F, \mu, \eta)$  a monad its algebras and homomorphisms thereof form a category we denote by  $\text{Alg}_{\text{Mnd}}(F)$ .



**Lemma B.7.** Suppose  $F$  is an endofunctor on a category  $\mathcal{C}$  with coproducts. Then there is an equivalence of categories  $\text{Alg}_{\text{Endo}}(F) \xrightarrow{\sim} \text{Alg}_{\text{Pendo}}((F + id_{\mathcal{C}}))$ .

**Definition B.8.** Given an endofunctor  $F : \mathcal{C} \rightarrow \mathcal{C}$ , an **algebraically free monad** on  $F$  is a monad  $\text{Free}_{\text{Mnd}}(F)$  together with an equivalence of categories  $\text{Alg}_{\text{Endo}}(F) \xrightarrow{\sim} \text{Alg}_{\text{Mnd}}(\text{Free}_{\text{Mnd}}(F))$  which preserves the respective functors to  $\mathcal{C}$  that forget the algebraic structure.

## B.2. Kelly's unified transfinite construction

The existence theorem for algebraically free monads is Kelly's unified transfinite construction (Kelly, 1980).

**Definition B.9** (Reflective subcategory). A subcategory  $\mathcal{D}$  of a category  $\mathcal{C}$  is **reflective** if the inclusion functor  $F : \mathcal{D} \rightarrow \mathcal{C}$  admits a left adjoint  $G : \mathcal{C} \rightarrow \mathcal{D}$ . A reflective subcategory of a presheaf category is called a **locally presentable category**.

**Example B.10** (Ubiquity of local presentability). Nearly every category you will encounter in practice is a locally presentable category. While its beyond the scope of this document to expound too much upon it, local presentability is a particularly powerful notion of what it means for objects and morphisms to be of things defined by equalities of set-sized expressions. The categories of monoids, groups, rings, vector spaces, and modules are all reflective subcategories of  $\text{Set}$ , which is a presheaf category. As are the categories of topological spaces, manifolds, metric spaces, and uniform spaces are likewise all reflective subcategories of  $\text{Set}$ . The category of small categories is a reflective subcategory of the category of graphs, which is itself a presheaf category.

**Definition B.11** (Limit). For categories  $\mathcal{J}$  and  $\mathcal{C}$ , a **diagram of shape  $\mathcal{J}$**  in  $\mathcal{C}$  is a functor  $D : \mathcal{J} \rightarrow \mathcal{C}$ . A **cone** to a diagram  $D$  consists of an object  $C \in \mathcal{C}$  and a family of morphisms  $c_j : C \rightarrow D(j)$  for each object  $j \in \mathcal{J}$ , such that for any  $f : i \rightarrow j$  in  $\mathcal{J}$  the following diagram commutes:

$$\begin{array}{ccc} C & \xrightarrow{c_i} & D(i) \\ & \searrow c_j & \downarrow D(f) \\ & & D(j) \end{array}$$

A morphism of cones  $\theta : (C, c_j) \rightarrow (C', c'_j)$  is a morphism  $\theta$  in  $\mathcal{C}$  making each diagram

$$\begin{array}{ccc} C & \xrightarrow{\theta} & C' \\ & \searrow c_j & \downarrow c'_j \\ & & D(j) \end{array}$$

commute. The **limit** of a diagram  $D$ , written  $\lim D$ , is the terminal object in the category  $\text{Cone}(D)$  of cones to  $D$  and morphisms between them.

**Definition B.12** (Colimit). The **colimit**  $\lim_{\rightarrow} D$  of a diagram  $D : \mathcal{J}^{op} \rightarrow \mathcal{C}$  is the initial object in the category  $\text{Cocone}(D)$  of cocones to  $D$ , where a cocone  $(C, c_j : D(j) \rightarrow C)$  has the dual property of a cone (above) with the morphisms reversed. **Small** (respectively  $\kappa$ -directed, connected, ...) colimits are colimits for which the indexing category  $\mathcal{J}$  is small (respectively  $\kappa$ -directed, connected, ...).

**Example B.13.** A terminal object in  $\mathcal{C}$  is a limit of the unique diagram from the empty category to  $\mathcal{C}$ . Similarly an initial object is an example of a colimit.

**Definition B.14** (Accessible category and accessible functor). For an ordinal  $\kappa$ , a  **$\kappa$ -accessible category**  $\mathcal{C}$  is a category such that:

- $\mathcal{C}$  has  $\kappa$ -directed colimits; and
- there is a set of  $\kappa$ -compact objects which generates  $\mathcal{C}$  under  $\kappa$ -directed colimits.

A  **$\kappa$ -accessible functor**  $F : \mathcal{C} \rightarrow \mathcal{D}$  is an functor between  $\kappa$ -accessible functor which preserves  $\kappa$ -filtered colimits.

**Remark B.15.** The accessibility of a functor can be thought of as an upper-bound on the arity of the operations which it abstracts. For example finite sums of finite sums are again finite sums.

**Definition B.16.** Let  $\mathcal{C}$  be a  $\kappa$ -accessible locally presentable category and  $F : \mathcal{C} \rightarrow \mathcal{C}$  a pointed  $\kappa$ -accessible endofunctor. Let  $F^\kappa$  be the  $\kappa$ -directed colimit of the diagram

$$F^0 \rightarrow F^1 \rightarrow F^2 \rightarrow \dots \rightarrow F^\kappa$$

where  $F^0 = \text{id}_{\mathcal{C}}$  and  $F^{\alpha+1} = F \circ F^\alpha$  for  $\alpha < \kappa$  and  $F^\alpha = \varinjlim_{\beta < \alpha} F^\beta$  for  $\alpha$  a limit ordinal.

**Lemma B.17.** For  $\mathcal{C}$  and  $F$  as above,  $F^\kappa$  is a monad. The unit is the canonical inclusion of  $\text{id}_{\mathcal{C}}$  into the colimit  $F^\kappa$  and the multiplication comes from the preservation of  $\kappa$ -filtered colimits by  $F$ .

**Theorem B.18.** Assume the hypotheses of Lemma B.17 with  $\kappa$  the ordinal in those hypotheses. Then  $\text{Alg}_{\text{Pendo}}(F)$  is equivalent to  $\text{Alg}_{\text{Mnd}}(F^\kappa)$  - i.e.  $F^\kappa$  is an algebraically free monad for  $F$ .

**Remark B.19.** For the endofunctors  $F$  we study here,  $\text{Free}(F) \xrightarrow{\sim} (F + \text{id})^\omega$  is the underlying endofunctor of an algebraically free monad for  $F$ ,  $\text{Free}(F)$ .

Free monads and cofree monads can be computed by the following explicit formula (Ghani et al.).

**Proposition B.20** ((Co)free (co)monads, explicitly). Let  $F : \mathcal{C} \rightarrow \mathcal{C}$  be an endofunctor. Then  $\text{Free}(F)$ , the free monad on  $F$  is given by  $\text{Free}(F)(Z) = \text{Fix}(X \mapsto F(X) + Z)$ . Dually, we compute  $\text{Cofree}(F)$ , the cofree comonad on  $F$  as  $\text{Cofree}(F)(Z) = \text{Fix}(X \mapsto F(X) \times Z)$ .

**Example B.21** (Free monad on  $1 + A \times -$ ). Free monad on  $1 + A \times -$  is  $\text{List}_{\neq 1}(A) : \text{Set} \rightarrow \text{Set}$ , the endofunctor mapping an object  $Z$  to the set  $\text{List}(Z)(A)$  of lists of elements of type  $A$  whose last element is not  $\square$ , i.e. an element of type 1, but instead an element of type  $Z + 1$ . That is, these are lists which end with potentially an element of  $Z$ .

**Example B.22** (Free monad on  $A + (-)^2$ ). Free monad on  $A + (-)^2$  is  $\text{Tree}(A + -) : \text{Set} \rightarrow \text{Set}$  which maps a set  $Z$  to the set of trees with  $A + Z$  labelled leaves.

**Example B.23** (Cofree comonad on  $(I \rightarrow O \times -)$ ). The cofree comonad on  $(I \rightarrow O \times -)$  is a hybrid type of Moore and Mealy machines  $\text{Fix}(X \mapsto (I \rightarrow O \times X) \times Z)$  which can be thought of as a hybrid of moore and mealy machines, outputting an additional element of  $Z$  at each step which does not depend on  $I$ .

## C. Lawvere Theories and Syntax

(de Haan et al., 2020a) expanded the theory of equivariant layers in neural networks using the abstraction of natural transformations of functors. In this section, we will explain how to understand morphisms of monad algebras in the same terms.

Indeed, this comparison is crucial to understanding the *syntax* of monads, in addition to the *semantics* given by their category of algebras.

**Definition C.1.** If  $\mathcal{C}$  is a category, a **presheaf on  $\mathcal{C}$**  is a functor  $\mathcal{C}^{op} \rightarrow \text{Set}$ . The category of presheaves with natural transformations is denoted  $\text{Psh}(\mathcal{C})$ .

Fix a monad  $T$  on  $\text{Set}$  and let  $\mathcal{C}_T$  denote its category of algebras.

Given an algebra  $A \in \mathcal{C}_T$ , the easiest way to interpret  $A$  as a functor is via the *Yoneda embedding*  $\mathcal{C}_T \rightarrow \text{Psh}(\mathcal{C}_T)$ , which identifies  $A$  with the presheaf  $[-, A]$ . It is a standard result that the Yoneda functor is fully faithful, which means that we can identify morphisms as algebras with morphisms as presheaves.

Furthermore, these presheaves have a special property. If  $j$  is a small colimit in  $\mathcal{C}_T$ , then  $[\varinjlim j, A] = \varprojlim [j, A]$ , essentially by the definition of limits and colimits.

**Definition C.2.** If  $\mathcal{C}$  is a category and  $J$  is a class of small colimits in  $\mathcal{C}$ , then a  **$J$ -continuous presheaf on  $\mathcal{C}$**  is a presheaf  $F \in \text{Psh}(\mathcal{C})$  satisfying  $F(\varinjlim j) = \varprojlim F(j)$  for all  $j \in J$ . The corresponding full subcategory of  $\text{Psh}(\mathcal{C})$  is denoted  $\text{CPsh}_J(\mathcal{C})$ , or simply  $\text{CPsh}(\mathcal{C})$  in the case that  $J$  is the class of all small colimits. We will also use  $\coprod$  to refer to the class of all coproducts,  $+$  to refer to the class of binary coproducts, and  $\emptyset$  to refer to the empty class.

It turns out that  $\mathcal{C}_T$  has a nice property: it is a *strongly compact* category, meaning that every continuous presheaf is representable as above by an object of  $\mathcal{C}_T$ . In other words, we have the identification  $\mathcal{C}_T = \text{CPsh}(\mathcal{C}_T)$ .

However, it is usually impractical to work with the entire category  $\mathcal{C}_T$ . When possible, we want to reason in terms of a more tractable subcategory. These will provide us with workable *syntax* for our monad. Following (Brandenburg, 2021),

we make use of Ehresmann’s concept of a “colimit sketch”: a category equipped with a restricted class of colimits. Rather than giving the general definition, we will describe a few special cases of prime interest.

**Definition C.3.** The **Kleisli category**  $\mathcal{K}_T$  is the full subcategory of  $\mathcal{C}_T$  on the free algebras. The **finitary Kleisli category**  $\mathcal{K}_T^{\mathbb{N}}$  is the full subcategory on the free algebras of the form  $TS$  for  $S$  a finite set. And the **unary Kleisli category**  $\mathcal{K}_T^1$  is the full one-object subcategory on the free algebra  $T1$ .

We have the following sequence of nested full subcategories:

$$\mathcal{C}_T \supset \mathcal{K}_T \supset \mathcal{K}_T^{\mathbb{N}} \supset \mathcal{K}_T^1$$

By composing with these inclusions, we get a sequence of functors between presheaf categories:

$$\text{Psh}(\mathcal{C}_T) \rightarrow \text{Psh}(\mathcal{K}_T) \rightarrow \text{Psh}(\mathcal{K}_T^{\mathbb{N}}) \rightarrow \text{Psh}(\mathcal{K}_T^1)$$

By restricting our class of colimits, we can restrict this to the corresponding continuous presheaf categories:

$$\mathcal{C}_T \cong \text{CPsh}(\mathcal{C}_T) \rightarrow \text{CPsh}_{\text{II}}(\mathcal{K}_T) \rightarrow \text{CPsh}_+(\mathcal{K}_T^{\mathbb{N}}) \rightarrow \text{CPsh}_{\emptyset}(\mathcal{K}_T^1)$$

It turns out that in many cases, these arrows are equivalences of categories.

**Definition C.4.** In the case that  $\mathcal{C}_T \rightarrow \text{CPsh}_{\text{II}}(\mathcal{K}_T)$  is an equivalence, we say that  $\mathcal{K}_T^{op}$  is the **infinitary Lawvere theory** for  $T$ .

*Remark C.5.* In fact, all monads have an infinitary Lawvere theory. For a proof, see (Brandenburg, 2021).

**Definition C.6.** In the case that  $\mathcal{C}_T \rightarrow \text{CPsh}_+(\mathcal{K}_T^{\mathbb{N}})$  is an equivalence, we say that  $(\mathcal{K}_T^{\mathbb{N}})^{op}$  is the **Lawvere theory** for  $T$ , and  $T$  is a **finitary monad**.

**Example C.7.** If  $T$  is the monad sending a set  $S$  to the free commutative semiring on  $S$ , then  $\mathcal{C}_T$  is the category of commutative semirings. Since the axioms for commutative semirings consist of equations with a finite number of variables,  $T$  has a Lawvere theory  $(\mathcal{K}_T^{\mathbb{N}})^{op}$ .

It is well known that this is the category of “finite polynomials”, see e.g. (Gambino & Kock, 2013). This connection was observed in (Dudzik & Veličković, 2022) to relate message passing in Graph Neural Networks to polynomial functors.

In fact, we could further restrict this theory to just the four objects  $\{T0, T1, T2, T3\}$ , because we can fully axiomatise commutative semirings with only three variables, e.g.  $a(b + c) = ab + ac$ .

**Example C.8 (Monoids).** If  $\mathcal{C}_T \rightarrow \text{CPsh}_{\emptyset}(\mathcal{K}_T^1)$  is an equivalence, then in fact  $\mathcal{C}_T \cong \text{Psh}(\mathcal{K}_T^1)$ , which is the category of  $M$ -sets for the monoid  $M = (\mathcal{K}_T^1)^{op}$ . So we can see that “unary Lawvere theories” exactly correspond to monads of the form  $M \times -$ , where  $M$  is a monoid.

## D. Monoidal categories

**Definition D.1** (Strict monoidal category, (Johnson et al., Def. 1.2.1)). Let  $\mathcal{M}$  be a category. We call  $\mathcal{M}$  a *strict monoidal category* if it is equipped with the following data a functor  $\otimes : \mathcal{M} \times \mathcal{M} \rightarrow \mathcal{M}$  called *the monoidal product*; an object  $I \in \mathcal{M}$  called *the monoidal unit*, such that

- $A \otimes (B \otimes C) = (A \otimes B) \otimes C$  for all  $A, B, C \in \mathcal{M}$ ;
- $A \otimes I = A = I \otimes A$  for all  $A \in \mathcal{M}$ ;
- $f \otimes (g \otimes h) = (f \otimes g) \otimes h$  for all  $f, g, h \in \mathcal{M}$ ;
- $\text{id} \otimes f = f = f \otimes \text{id}$  for all  $f \in \mathcal{M}$ .

## E. Actegories

**Definition E.1** (Actegories, see (Capucci & Gavranović, 2023)). Let  $(\mathcal{M}, \otimes, I)$  be a monoidal category. A  $\mathcal{M}$ -actegory is  $\mathcal{C}$  is a category  $\mathcal{C}$  together with a functor  $\blacktriangleright: \mathcal{M} \times \mathcal{C} \rightarrow \mathcal{C}$  together with natural isomorphisms  $\eta_X: X \cong I \blacktriangleright X$  and  $\mu_{M,N}: (M \otimes N) \blacktriangleright X \cong M \blacktriangleright (N \blacktriangleright X)$ , such that:

- **Pentagonator.** For all  $M, N, P \in \mathcal{M}$  and  $C \in \mathcal{C}$  the following diagram commutes.

$$\begin{array}{ccc}
 C \blacktriangleright (M \otimes (N \otimes P)) & \xrightarrow{\mu_{C,M,N \otimes P}} & (C \blacktriangleright M) \blacktriangleright (N \otimes P) \xrightarrow{\mu_{C \blacktriangleright M, N, P}} ((C \blacktriangleright M) \blacktriangleright N) \blacktriangleright P \\
 \downarrow C \blacktriangleright \alpha_{M,N,P}^{-1} & & \parallel \\
 C \blacktriangleright ((M \otimes N) \otimes P) & \xrightarrow{\mu_{C,M \otimes N, P}} & (C \blacktriangleright (M \otimes N)) \blacktriangleright P \xrightarrow{\mu_{C,M,N} \blacktriangleright P} ((C \blacktriangleright M) \blacktriangleright N) \blacktriangleright P
 \end{array} \quad (7)$$

- **Left and right unitors.** For all  $C \in \mathcal{C}$  and  $M \in \mathcal{M}$  The diagram below commutes.

$$\begin{array}{ccc}
 C \blacktriangleright (M \otimes I) & & C \blacktriangleright (I \otimes M) \\
 \downarrow \mu_{C,M,I} & \searrow C \blacktriangleright \rho_M & \swarrow C \blacktriangleright \lambda_M \\
 (C \blacktriangleright M) \blacktriangleright I & \xrightarrow{\eta_{C \blacktriangleright M}} & C \blacktriangleright M \\
 & & \nwarrow \eta_{C \blacktriangleright M} \\
 & & (C \blacktriangleright I) \blacktriangleright M \\
 & & \downarrow \mu_{C,I,M}
 \end{array} \quad (8)$$

*Remark E.2.* provides that, just as one may assume a monoidal category to be strict, via MacLane's coherence theorem, we may assume actegories are strict as well (Capucci & Gavranović, 2023, Remark 3.4). That is to say we may assume the for an  $\mathcal{M}$  actegory  $(\mathcal{C}, \blacktriangleright)$  that:

- $I \blacktriangleright (-) = \text{id}_{\mathcal{C}}: \mathcal{C} \rightarrow \mathcal{C}$ ; and
- $\mu_{M,N}: ((-) \otimes (-) \blacktriangleright (-) = (-) \blacktriangleright (-) \blacktriangleright (-): \mathcal{M} \times \mathcal{M} \times \mathcal{C} \rightarrow \mathcal{C}$

We will call such a structure a **strict actegory**. We will use these later on to simplify exposition and some few theorems.

**Example E.3** (Monoidal action). Any monoidal category gives rise to a self-action.

**Example E.4** (Families actegories). Any category  $\mathcal{C}$  with coproducts has an action  $\blacktriangleright: \text{Set} \times \mathcal{C} \rightarrow \mathcal{C}$  which maps  $(X, A)$  to the coproduct of  $|X|$  copies of  $A$ .

### E.1. Morphisms of actegories

**Definition E.5** (Actegorical strong monad). Let  $(\mathcal{C}, \blacktriangleright)$  be a  $\mathcal{M}$ -actegory. A monad  $(T, \mu, \eta)$  on  $\mathcal{C}$  is called **strong**<sup>9</sup> if it is equipped with a natural transformation  $\sigma_{P,A}: P \blacktriangleright T(A) \rightarrow T(P \blacktriangleright A)$ , called **strength** such that diagrams in Definition E.8 commute.

**Example E.6.** All monads of the form  $A \times -: \text{Set} \rightarrow \text{Set}$  are strong for the the actegory  $(\text{Set}, \times)$ , for any monoid  $A$ . This includes  $G \times -: \text{Set} \rightarrow \text{Set}$  from Example 2.2.<sup>10</sup>

**Definition E.7** (Actegorical strong endofunctor). Let  $(\mathcal{C}, \blacktriangleright)$  be a  $\mathcal{M}$ -actegory. An endofunctor  $F$  on  $\mathcal{C}$  is called **strong** if it is equipped with a natural transformation  $\sigma_{P,A}: P \blacktriangleright F(A) \rightarrow F(P \blacktriangleright A)$ , called a **strength** such that diagrams **AS1** and **AS2** in Definition E.8 commute.

**Definition E.8** (Actegorical strong monad coherence diagrams). A monad  $(T, \mu, \eta)$  on the category  $\mathcal{C}$  of a  $\mathcal{M}$ -actegory  $(\mathcal{C}, \blacktriangleright)$  is called **strong** if it is equipped with a natural transformation  $\sigma_{P,A}: P \blacktriangleright T(A) \rightarrow T(P \blacktriangleright A)$ , called **strength** making the diagrams below commute:

<sup>9</sup>Another name for this is a  $\mathcal{M}$ -linear morphism, used in (Capucci & Gavranović, 2023)

<sup>10</sup>The strength is in fact a natural isomorphism.

**AS1: Compatibility of strength and the monoidal unit.**

$$\begin{array}{ccc} I \blacktriangleright T(A) & \xrightarrow{\sigma_{I,A}} & T(I \blacktriangleright A) \\ & \searrow \lambda_{T(A)} & \downarrow T(\lambda_A) \\ & & T(A) \end{array}$$

**AS2: Compatibility of strength and actegory multiplication.**

$$\begin{array}{ccc} (P \otimes Q) \blacktriangleright T(X) & \xrightarrow{\alpha_{P,Q,T(X)}} & P \blacktriangleright (Q \blacktriangleright T(X)) \\ \downarrow \sigma_{P \otimes Q, X} & & \downarrow P \blacktriangleright \sigma_{Q, X} \\ T((P \otimes Q) \blacktriangleright X) & \xrightarrow{T(\alpha_{P,Q,X})} & T(P \blacktriangleright (Q \blacktriangleright X)) \\ & & \downarrow \sigma_{P, Q \blacktriangleright X} \\ & & P \blacktriangleright T(Q \blacktriangleright X) \end{array}$$

**AS3: Compat. of strength and monad multiplication.**

$$\begin{array}{ccc} A \blacktriangleright T(T(X)) & \xrightarrow{A \blacktriangleright \mu_X} & A \blacktriangleright T(X) \\ \downarrow \sigma_{A, T(X)} & & \downarrow \sigma_{A, X} \\ T(A \blacktriangleright T(X)) & & \\ \downarrow T(\sigma_{A, T(X)}) & & \\ T(T(A \blacktriangleright X)) & \xrightarrow{\mu_{A \blacktriangleright X}} & T(A \blacktriangleright X) \end{array}$$

**AS4: Compatibility of strength and monad unit.**

$$\begin{array}{ccc} & M \blacktriangleright X & \\ M \blacktriangleright \eta_X \swarrow & & \searrow \eta_{M \blacktriangleright X} \\ M \blacktriangleright T(X) & \xrightarrow{\sigma_{M, X}} & T(M \blacktriangleright X) \end{array}$$

**Example E.9.** The examples of endofunctors from Examples 2.9 to 2.11, I.4 and I.7 are all strong endofunctors, and their appropriate free monads are strong too. See Appendix (Example E.10)

**Example E.10.** Below we present a list the data of *strengths*  $\sigma_{P,X}$  of some relevant endofunctors and (on the left) and their corresponding free monads (on the right).

Given the endofunctor  $1 + A \times - : \text{Set} \rightarrow \text{Set}$ , its strength  $\sigma_{P,X} : P \times (1 + A \times X) \rightarrow 1 + A \times P \times X$  is given by:

$$\begin{aligned} \sigma_{P,X}(p, \text{inl}(\bullet)) &= \text{inl}(\bullet) \\ \sigma_{P,X}(p, \text{inr}(a, x)) &= \text{inr}(a, p, x) \end{aligned}$$

Given the endofunctor  $A + (-)^2$ , its strength  $\sigma_{P,X} : P \times (A + (P \times X)^2) \rightarrow A + (P \times X)^2$  is given by:

$$\begin{aligned} \sigma_{P,X}(p, \text{inl}(a)) &= \text{inl}(a) \\ \sigma_{P,X}(p, \text{inr}(x, x')) &= \text{inr}((p, x), (p, x')) \end{aligned}$$

Given the free monad  $\text{List}_{+1}(A) : \text{Set} \rightarrow \text{Set}$ , its strength  $\sigma_{P,Z} : P \times \text{List}_{Z+1}(A) \rightarrow \text{List}_{P \times Z+1}(A)$  is given by:

$$\begin{aligned} \sigma_{P,Z}(p, \text{Nil}) &= \text{Nil} \\ \sigma_{P,Z}(p, z) &= (p, z) \\ \sigma_{P,Z}(p, \text{Cons}(a, as)) &= \text{Cons}(a, \sigma_{P,Z}(p, as)) \end{aligned}$$

Given the free monad  $\text{Tree}(A + -) : \text{Set} \rightarrow \text{Set}$ , its strength  $\sigma_{P,Z} : P \times \text{Tree}(A + Z) \rightarrow \text{Tree}(A + P \times Z)$  is given by:

$$\begin{aligned} \sigma_{P,Z}(p, \text{Leaf}(\text{inl}(a))) &= \text{Leaf}(a) \\ \sigma_{P,Z}(p, \text{Leaf}(\text{inr}(z))) &= \text{Leaf}(\text{inr}(p, z)) \\ \sigma_{P,Z}(p, \text{Node}(l, r)) &= \text{Node}(\sigma_{P,Z}(p, l), \sigma_{P,Z}(p, r)) \end{aligned}$$

## F. 2-Categorical algebra

A good reference for 2-categorical algebra is (Lack, 2010) or (Kelly, 2005), which deals with the more general notion of enriched categories which 2-categories are a particular example of.

### F.1. 2-monads and their lax algebras

A 2-monad is a monad is a Cat-enriched monad. More properly

**Definition F.1** (2-monad). A **2-monad** on a 2-category  $\mathcal{C}$  comprises:

- A 2-endofunctor  $T$  on  $\mathcal{C}$ ;
- A 2-natural transformation  $\mu : T^2 \Rightarrow T$ ; and



- A 2-natural transformation  $\eta : \text{id}_{\mathcal{C}} \Rightarrow T$ ;

such that the (Cat enriched variant of) the axioms Definition B.1 hold.

**Definition F.2** (Lax  $T$ -algebra for a 2-monad). Let  $(T, \eta, \mu)$  be a 2-monad on  $\mathcal{B}$ . A lax  $T$ -algebra is a pair  $(A, r, \epsilon_A, \delta_A)$  where  $A$  is an object of  $\mathcal{B}$ ,  $a : T(A) \rightarrow A$  is a morphism in  $\mathcal{B}$ , and  $\epsilon_A$  and  $\delta_A$  are the following 2-morphisms in  $\mathcal{B}$ :

$$\begin{array}{ccc} T(A) & & T(T(A)) \xrightarrow{\mu_A} T(A) \\ a \downarrow & \nwarrow \eta_A & \delta_A \nearrow \\ A & \xrightarrow{\epsilon_A} & T(A) \xrightarrow{a} A \end{array}$$

such that the lax unity and lax associativity conditions (Johnson et al., Eq. 6.5.6 and 6.5.7) are satisfied.

**Example F.3** (2-monad for  $M$ -modules). If  $M$  is a monoid, it is useful to study “ $M$ -modules”, which are monoids equipped with a compatible action of  $M$ . For example, the situation of a group acting on a vector space is fundamental in representation theory. We can interpret such modules in terms of the 2-monad  $BM \times -$  on the 2-category Cat, where  $BM$  is the one-object category with arrows  $M$ . Then an  $M$ -module is seen to be a one-object algebra for this 2-monad. Indeed, this is just a special case of an actegory.

**Definition F.4** (Lax algebra homomorphism). Let  $\mathcal{B}$  be a 2-category,  $T$  a 2-monad on  $\mathcal{B}$ , and  $(A, a, \epsilon_A, \delta_A)$  and  $(B, b, \epsilon_B, \delta_B)$  be lax  $T$ -algebras. Then a lax algebra morphism from  $(A, a, \epsilon_A, \delta_A) \rightarrow (B, b, \epsilon_B, \delta_B)$  is pair  $(f, \kappa)$  where  $f : A \rightarrow B$  is a morphism in  $\mathcal{B}$  and  $\kappa$  is a 2-morphism spanned by the following diagram

$$\begin{array}{ccc} T(A) & \xrightarrow{T(f)} & T(B) \\ a \downarrow & \nwarrow \kappa & \downarrow b \\ A & \xrightarrow{f} & B \end{array}$$

such that the diagrams in (Johnson et al., Def. 6.5.9) commute.

Lax algebras and lax algebra homomorphisms for the 2-monad  $T$  form the category  $\text{Lax-Alg}_{\text{Mnd}}(T)$ .

**Example F.5** (Cocycles as lax morphisms). The monoid cocycles, or “crossed homomorphisms”, used in (Dudzic et al., 2023) to study asynchrony in algorithms and networks, can be described as lax morphisms for 2-monad algebras. As above, if  $A$  is an  $M$ -module, then  $BA$  is an algebra for  $BM \times -$ . A lax morphism  $1 \rightarrow BA$  is a natural transformation of the identity functor  $BM \rightarrow BA$ , which is just a set map  $D : M \rightarrow A$ . The axiom for compositionality is exactly the 1-cocycle condition for  $D$ , so 1-cocycles are the same as lax morphisms  $1 \rightarrow BA$ .

## G. The 2-category $\text{Para}(\mathcal{C})$

**Definition G.1** (Para, compare (Crutwell et al., 2022; Capucci et al., 2022)). Given a monoidal category  $(\mathcal{M}, \otimes, I)$  and an  $\mathcal{M}$ -actegory  $\mathcal{C}$ , let  $\text{Para}_{\blacktriangleright}(\mathcal{C})$  be the 2-category whose:

- Objects are the objects of  $\mathcal{C}$ ;
- Morphisms  $X \rightarrow Y$  are pairs  $(P, f)$ , where  $P$  is an object of  $\mathcal{M}$  and  $f : P \blacktriangleright X \rightarrow Y$  is a morphism of  $\mathcal{C}$ ;

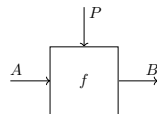


Figure 2. String diagram representation of a parametric morphism. We often draw the parameter wire on the *vertical* axis to signify that the parameter object is part of the data of the morphism.

- 2-morphisms  $(P, f) \Rightarrow (P', f') : X \rightarrow Y$  are 1-morphisms  $r : P' \rightarrow P$  of  $\mathcal{M}$  such that the triangle

$$\begin{array}{ccc}
 P \triangleright X & & \\
 \uparrow r \triangleright X & \searrow f & \\
 P' \triangleright X & \nearrow f' & Y
 \end{array}$$

commutes (equivalently an equality of parametric string diagrams as in Figure 3).

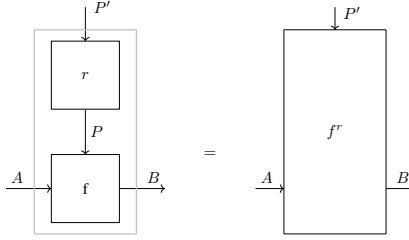


Figure 3. String diagram of reparameterisation. The reparameterisation map  $r$  is drawn vertically.

- Identity morphism on  $X$  is the parametric map  $(I, \eta_X^{-1})$ , where  $\eta$  is the unitor of the underlying actegory.

where composition of morphisms  $(P, f) : X \rightarrow Y$  and  $(Q, g) : Y \rightarrow Z$  is  $(Q \otimes P, h)$  where  $h$  is the composite

$$(Q \otimes P) \blacktriangleright X \xrightarrow{\mu_{Q,P,X}} Q \blacktriangleright (P \blacktriangleright X) \xrightarrow{Q \blacktriangleright f} Q \blacktriangleright Y \xrightarrow{g} Z$$

and composition of 2-morphisms is given by the composition of  $\mathcal{M}$ .

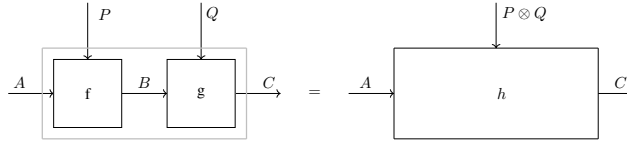


Figure 4. String diagram representation of the composition of parametric morphisms. By treating parameters on a separate axis we obtain an elegant graphical depiction of their composition.

**Remark G.2.** When the actegory is strict the identities of the actegory reduce to being the parametric morphisms of the form  $(I, \text{id}_X)$

**Example G.3 (Real Vector Spaces and Smooth Maps).** Consider the cartesian category **Smooth** whose objects are real vector spaces, and morphisms are smooth functions. As this category is cartesian, we can form  $\text{Para}(\text{Smooth})$  modelling parametric smooth functions.

**Lemma G.4** (Embedding of  $\mathcal{C}$  into  $\text{Para}(\mathcal{C})$ ). *There is a 2-functor  $\gamma : \mathcal{C} \rightarrow \text{Para}(\mathcal{C})$  which is identity-on-objects and treats every morphism  $f : A \rightarrow B$  as trivially parametric, i.e. as  $(I, f \circ \eta_A^{-1})$ .<sup>11</sup>*

**Remark G.5.** When the actegory  $(\mathcal{C}, \blacktriangleright)$  is strict the trivially parametric functions do not require precomposition with the unitor of actegory. That is to say a morphism  $f : X \rightarrow Y$  of  $\mathcal{C}$  is sent to the parametric morphism  $(I, f) : X \rightarrow Y$ .

**Theorem G.6** ( $\gamma$  preserves connected colimits). *Suppose  $(\mathcal{M}, \eta, \mu)$  is a strict monoidal category, that  $(\mathcal{C}, \blacktriangleright)$  is an  $\mathcal{M}$ -actegory, and that  $X \blacktriangleright (-) : \mathcal{C} \rightarrow \mathcal{C}$  preserves connected colimits. Then, the 2-functor  $\gamma : \mathcal{C} \rightarrow \text{Para}(\mathcal{C})$  preserves connected colimits.*

<sup>11</sup>Here we are treating  $\mathcal{C}$  as 2-category with only identity 2-morphisms.

*Remark G.7.* While in  $\mathcal{C}$  they are the only kind of colimits, the colimits we mean in  $\text{Para}(\mathcal{C})$  are strict colimits. The relationship to other flavours of colimit is a topic of ongoing research.

*Proof.* Note that for a cone over a trivially parameterised connected diagram to commute, the parameters of the 1-cells of the cone must all be the same. Then, since  $\blacktriangleright$  preserves connected colimits by hypothesis, we may assemble the following chain of isomorphisms.

$$\begin{aligned}
 \text{Tr}_1(\text{Para}(\mathcal{C}))\left(\varinjlim_{d \in \mathcal{D}} X_d, Y\right) &\xrightarrow{\sim} \coprod_{P \in \mathcal{M}} \mathcal{C}\left(P \triangleright \varinjlim_{d \in \mathcal{D}} X_d, Y\right) \\
 &\xrightarrow{\sim} \coprod_{P \in \mathcal{M}} \mathcal{C}\left(\varinjlim_{d \in \mathcal{D}} P \triangleright X_d, Y\right) \\
 &\xrightarrow{\sim} \coprod_{P \in \mathcal{M}} \varprojlim_{d \in \mathcal{D}} \mathcal{C}(P \triangleright X_d, Y) \\
 &\xrightarrow{\sim} \text{Ob}(\mathcal{M}) \times \varprojlim_{d \in \mathcal{D}} \mathcal{C}(P \triangleright X_d, Y) \\
 &\xrightarrow{\sim} \varprojlim_{d \in \mathcal{D}} (\text{Ob}(\mathcal{M}) \times \mathcal{C}(P \triangleright X_d, Y)) \\
 &\xrightarrow{\sim} \varprojlim_{d \in \mathcal{D}} \left( \coprod_{P \in \mathcal{M}} \mathcal{C}(P \triangleright X_d, Y) \right) \\
 &\xrightarrow{\sim} \varprojlim_{d \in \mathcal{D}} (\text{Tr}_1(\text{Para}(\mathcal{C}))(X_d, Y))
 \end{aligned}$$

□

**Example G.8.** Fix  $\mathcal{M}$ -actegory  $(\mathcal{C}, \blacktriangleright)$ , and a monad  $(T, \mu, \eta)$  on  $\mathcal{C}$  with actegorical strength  $\sigma : M \blacktriangleright T(X) \rightarrow T(M \blacktriangleright X)$ . Then this gives us a 2-monad on  $\text{Para}_{\blacktriangleright}(T)$  on  $\text{Para}_{\blacktriangleright}(\mathcal{C})$  whose underlying 2-endofunctor:

- Acts as  $T$  does on objects<sup>12</sup>; and
- Sends  $(P, f) : X \rightarrow Y$  to  $(P, f')$ , where  $f'$  is the composite

$$P \blacktriangleright T(X) \xrightarrow{\sigma_{P,X}} T(P \blacktriangleright X) \xrightarrow{T(f)} T(Y)$$

- Sends  $r : P \rightarrow P'$  to itself.

and the unit and multiplication 2-natural transformations are defined as follows:

- Unit  $\text{Para}_{\blacktriangleright}(\eta) : \text{id}_{\text{Para}_{\blacktriangleright}(\mathcal{C})} \Rightarrow \text{Para}_{\blacktriangleright}(T)$  whose component at  $X : \mathcal{C}$  is the element of  $\text{Para}_{\blacktriangleright}(\mathcal{C})(X, T(X))$  given by the parametric morphism  $(I, u_X)$ , where  $u_X$  is the composite

$$I \blacktriangleright X \xrightarrow{\eta_X^{-1}} X \xrightarrow{\eta_X^T} T(X)$$

For every parametric map  $(P, f) \in \text{Para}(\mathcal{C})(X, Y)$  we have the strictly natural square in Equation (9). The 2-cell is given by the reparameterisation  $\beta_{P,I} : P \otimes I \rightarrow I \otimes P$  which is identity since we are assuming  $\mathcal{M}$  is strict monoidal. It can be checked that this indeed satisfies the conditions of a 2-morphism in  $\text{Para}_{\blacktriangleright}(\mathcal{C})$ . This makes  $\text{Para}_{\blacktriangleright}(\eta)$  a 2-natural transformation.

$$\begin{array}{ccc}
 X & \xrightarrow{(P,f)} & Y \\
 (I,u) \downarrow & \swarrow \beta_{P,I} & \downarrow (I,u_Y) \\
 T(X) & \xrightarrow{T(f) \circ \sigma_{P,X}} & T(Y)
 \end{array} \tag{9}$$

<sup>12</sup>This is well defined since  $\text{Ob}(\text{Para}(\mathcal{C})) = \text{Ob}(\mathcal{C})$

- Multiplication  $\text{Para}_{\blacktriangleright}(\mu) : \text{Para}_{\blacktriangleright}(T)^2 \Rightarrow \text{Para}_{\blacktriangleright}(T)$  whose component at  $X \in \text{Para}_{\blacktriangleright}(\mathcal{C})$  is the parametric morphism  $(I, m_X)$  where  $m_X$  is the composite

$$I \blacktriangleright T^2(X) \xrightarrow{\eta_X^{\blacktriangleright^{-1}}} T^2(X) \xrightarrow{\mu_X^T} T(X)$$

For every parametric morphism  $(P, f) \in \text{Para}_{\blacktriangleright}(\mathcal{C})(X, Y)$  we have the strictly natural square in Equation (10). It is again given by  $\beta_{P,I}$ , and is strict because  $\mathcal{M}$  is strict monoidal. It can be checked that this too satisfies the conditions of a 2-morphism in  $\text{Para}_{\blacktriangleright}(\mathcal{C})$ . This makes  $\text{Para}_{\blacktriangleright}(\mu)$  a 2-natural transformation.

$$\begin{array}{ccc} T^2(X) & \xrightarrow{(P, (T(f) \circ \sigma_{P,X}) \circ \sigma_{P,T(X)})} & T^2(Y) \\ (I, m_x) \downarrow & \swarrow \beta_{P,I} & \downarrow (I, m_y) \\ T(X) & \xrightarrow{T(f) \circ \sigma_{P,X}} & T(Y) \end{array} \quad (10)$$

Lastly, we need to check that this indeed satisfies the 2-monad coherence conditions. Since the parameters of the components of  $\text{Para}_{\blacktriangleright}(\eta)$  and  $\text{Para}_{\blacktriangleright}(\mu)$  are all trivial, this becomes straightforward to check.

*Remark G.9.* For strictly monoidal  $\mathcal{M}$  and a strict actegory  $(\mathcal{C}, \blacktriangleright)$  the unit and multiplication of  $\text{Para}(T)$  are exactly the unit and multiplication of  $T$ .

**Theorem G.10** (Lax (co)algebras for  $\text{Para}_{\blacktriangleright}(T)$  induce comonoids). *Let  $(\mathcal{C}, \blacktriangleright)$  be a  $\mathcal{M}$ -actegory and  $T : \mathcal{C} \rightarrow \mathcal{C}$  a strong actegorical monad on  $\mathcal{C}$ . Consider a lax algebra  $(A, (P, a), \epsilon_A, \delta_A)$  for the induced 2-monad  $\text{Para}(T)$ . Then  $P$  is a comonoid in  $\mathcal{M}$  where  $\epsilon_A$  is the data of its counit, and  $\delta_A$  the data of its comultiplication, and the comonoid laws follow from lax algebra coherence conditions. Dually, the same statement holds for a lax coalgebra of a  $\text{Para}(T)$  comonad.*

*Proof.* We start by unpacking the data of the lax algebra 2-cells.

$$\begin{array}{ccc} & T(X) & \\ (P,a) \swarrow & \uparrow \epsilon_P & \nwarrow (I, \eta_X^T \circ \eta_X^{\blacktriangleright^{-1}}) \\ X & \xlongequal{(I, \lambda_X)} & X \end{array} \quad \begin{array}{ccc} T(T(X)) & \xrightarrow{(I, m_x)} & T(X) \\ (P, T(a) \circ \sigma_{P,X}) \downarrow & \nearrow \delta_P & \downarrow (P,a) \\ T(X) & \xrightarrow{(P,a)} & X \end{array}$$

We can see that they are given by two reparameterisations in  $\mathcal{M}$ :  $\epsilon_P : P \otimes I \rightarrow I$  in  $\mathcal{M}$  and  $\delta_P : I \otimes P \rightarrow P \otimes P$ . These uniquely determine the morphisms which we call  $!_P : P \rightarrow I$  and  $\Delta_P : P \rightarrow P \otimes P$  respectively. To show that this is indeed a comonoid, we need to unpack the lax algebra laws. We note that all the conditions unpack only to conditions on morphisms in  $\mathcal{M}$ . It is relatively tedious, but straightforward to check that these conditions ensure that  $!_P$  and  $\Delta_P$  satisfy the comonoid laws.  $\square$

**Corollary G.11.** *The functor  $\text{Lax-Alg}_{\text{Mnd}}(\text{Para}(T)) \rightarrow \text{CoMon}(\mathcal{C}) \times \text{Lax}(\rightarrow, \mathcal{C})$  which takes lax algebras for  $\text{Para}(T)$  to the pair of the underlying comonoid parameter and the  $\mathcal{C}$ -morphism which interprets the parametric structure map is full-and-faithful.*

*Proof.* The proof is formal and left to the reader  $\square$

**Conjecture G.12.** Given a cartesian monoidal category  $(\mathcal{M}, \times, \bullet)$ , an  $\mathcal{M}$ -actegory  $(\mathcal{C}, \blacktriangleright)$ , and a strong endofunctor  $F$  on  $\mathcal{C}$  such that  $F$  and  $\mathcal{C}$  satisfy the hypotheses of Lemma Theorem B.18, then the assignment

$$(X, (P, f)) \longmapsto (X, ((P, !_P : P \rightarrow \bullet, \Delta_P : P \rightarrow P \times P), \text{laxlim}((P, f) \circ \text{Para}(F)(P, f) \circ \dots \circ \text{Para}(F)^\alpha(P, f)) : F^\kappa X \rightarrow X))$$

defines an equivalence of categories

$$\text{Lax-Alg}_{\text{Endo}}(\text{Para}(F)) \rightarrow \text{Lax-Alg}_{\text{Mnd}}(F^\kappa)$$

## H. Weight tying in GDL examples

### I. Examples from geometric deep learning

**Example I.1** (Linear equivariant layers for a pair of pixels). Consider the category  $\mathbf{Vect}$  of finite-dimensional vector spaces and linear maps. For simplicity, we will assume that the carrier set for our data is  $\mathbb{R}^{\mathbb{Z}_2}$ , which is a pair of pixels. Consider a linear endofunction on such data,  $f_{\mathbf{W}} : \mathbb{R}^{\mathbb{Z}_2} \rightarrow \mathbb{R}^{\mathbb{Z}_2}$ . It is well known that this function can be represented as a multiplication with a  $2 \times 2$  matrix  $\mathbf{W} = \begin{bmatrix} w_1 & w_3 \\ w_2 & w_4 \end{bmatrix}$ .

Now consider the group of 1D translations  $(\mathbb{Z}_2, +, 0)$ —which in this case amounts to *pixel swaps*—and its induced action  $\blacktriangleright$  on  $\mathbb{R}^{\mathbb{Z}_2}$ . If  $f_{\mathbf{W}}$  is equivariant with respect to  $\blacktriangleright$ , then via Example 2.6, for any input  $[x_1, x_2] \in \mathbb{R}^{\mathbb{Z}_2}$  it has to hold that

$$\begin{bmatrix} w_1 x_2 + w_2 x_1 \\ w_3 x_2 + w_4 x_1 \end{bmatrix} = \begin{bmatrix} w_3 x_1 + w_4 x_2 \\ w_1 x_1 + w_2 x_2 \end{bmatrix} \quad (11)$$

This implies that  $w_3 = w_2$  and  $w_4 = w_1$ , meaning every  $\mathbf{W}$  is of the form

$$\begin{bmatrix} w_1 & w_2 \\ w_2 & w_1 \end{bmatrix}$$

where the weight tying makes the matrix a symmetric one. Any neural network  $f_{\mathbf{W}}$  satisfying this constraint will be a *linear translation equivariant layer* over  $\mathbb{Z}_2$ , and can be used as a building block to construct geometric deep learning architectures.

**Remark I.2** (Circulants and CNNs). Note that this concept generalises to larger input domains (e.g.  $\mathbb{R}^{\mathbb{Z}_k}$  for  $k > 2$ ). Generally, it is a well-known fact in signal processing that, for  $f_{\mathbf{W}}$  to satisfy a linear translation equivariance constraint,  $\mathbf{W}$  must be a *circulant* matrix. Circulant matrices are known in neural networks as *convolutional layers*, the essence of modern CNNs (Fukushima et al., 1983; LeCun et al., 1998).

**Example I.3** (Invariant maps). Invariant maps are also  $(G \times -)$ -algebra homomorphisms where the codomain is a trivial group action. Specifically, setting the domain to any of the group actions from Example 2.6, the corresponding induced commutative diagram is below:

$$\begin{array}{ccc} G \times \mathbb{R}^{\mathbb{Z}_w \times \mathbb{Z}_h} & \xrightarrow{G \times f} & G \times \mathbb{R}^{\mathbb{Z}_w \times \mathbb{Z}_h} \\ \blacktriangleright \downarrow & & \downarrow \pi_2 \\ \mathbb{R}^{\mathbb{Z}_w \times \mathbb{Z}_h} & \xrightarrow{f} & \mathbb{R}^{\mathbb{Z}_w \times \mathbb{Z}_h} \end{array}$$

Elementwise, the commutativity of that diagram unpacks to the equation

$$f(g \blacktriangleright x) = f(x)$$

The translation example, for instance, recovers the equation  $f(((i', j') \blacktriangleright x)(i, j)) = (i', j') \blacktriangleright f(x)(i, j)$  which reduces to  $f(x(i - i', j - j')) = f(x)(i, j)$ . Intuitively, it states that, for any displacement  $(i', j')$ , the result of translating the input by  $(i', j')$  and then applying the function  $f$  is the same as just applying  $f$ .

We can run the analogous calculation with weight sharing here. We consider the same linear endofunction  $f_{\mathbf{W}} \in \mathbb{R}^{\mathbb{Z}_2} \rightarrow \mathbb{R}^{\mathbb{Z}_2}$  on a pair of pixels (represented as a matrix  $W = \begin{bmatrix} w_1 & w_3 \\ w_2 & w_4 \end{bmatrix}$ ), and the same group of translations  $(\mathbb{Z}_2, +, 0)$  on the domain, but this time we consider the identity endomorphism on the codomain. If  $f_{\mathbf{W}}$  is invariant to  $\blacktriangleright$ , then Example I.3 for any input  $[x_1, x_2] \in \mathbb{R}^{\mathbb{Z}_2}$  it has to hold that

$$\begin{bmatrix} w_1 x_2 + w_2 x_1 \\ w_3 x_2 + w_4 x_1 \end{bmatrix} = \begin{bmatrix} w_3 x_1 + w_4 x_2 \\ w_1 x_1 + w_2 x_2 \end{bmatrix} \quad (12)$$

implying  $w_1 = w_2$  and  $w_3 = w_4$ , meaning the equivariance induced a particular weight tying scheme



$$\begin{bmatrix} w_1 & w_3 \\ w_1 & w_3 \end{bmatrix}$$

in which the matrix has shared rows. Note that this implies that each of the pixel values in the output of  $f_W$  will be the same, and hence this layer could also be represented as just a single dot product with  $[w_1 \ w_3]$ , *eliminating* the grid structure in the original set. Such a layer would not be an endofunction on  $\mathbb{R}^{\mathbb{Z}^2}$ , however, which was our initial space of exploration.

## I.1. Examples from automata theory

### I.1.1. STREAMS

**Example I.4** (Streams). Let  $O$  be a set, thought of as the set of outputs. Consider the endofunctor from  $O \times - : \text{Set} \rightarrow \text{Set}$ . Then the set  $\text{Stream}(O)$  of streams<sup>13</sup> with outputs  $O$ , together with the map  $\langle \text{output}, \text{next} \rangle : \text{Stream}(O) \rightarrow O \times \text{Stream}(O)$  forms a coalgebra of this endofunctor. They have a representation as the following datatype

---

```
data Stream o = MkStream {
  output :: o
  next   :: Stream o
}
```

---

This datatype describes streams *coinductively*, as something from which we can always extract an output, and another stream. In Example J.3 we will see how this will be related to unfolding recurrent neural networks.

**Example I.5** (Unfolds to streams as coalgebra homomorphisms). Consider the endofunctor  $(O \times -)$  from Example I.4, and a coalgebra homomorphism from any other  $(O \times -)$ -coalgebra  $(X, \langle o, n \rangle)$  into  $(\text{Stream}(O), \langle \text{output}, \text{next} \rangle)$ :

$$\begin{array}{ccc} X & \xrightarrow{f_{o,n}} & \text{Stream}(O) \\ \langle o, n \rangle \downarrow & & \downarrow \langle \text{output}, \text{next} \rangle \\ O \times X & \xrightarrow{O \times f_{o,n}} & O \times \text{Stream}(O) \end{array}$$

Then the map  $f_{o,n} : X \rightarrow \text{Stream}(O)$  is necessarily a *unfold* to a stream, a concept from functional programming describing how a stream of values can be obtained from a single value. It is implemented by *corecursion* on the input:

---

```
fo,n :: x -> Stream o
fo,n x = MkStream (o x) (fo,n (n x))
```

---

This corecursion is structural in nature, meaning it satisfies the following two equations which arise by unpacking the algebra homomorphism equations elementwise:

$$o(x) = \text{output}(f_{o,n}(x)) \tag{13}$$

$$f_{o,n}(n(x)) = \text{next}(f_{o,n}(x)) \tag{14}$$

Here Equation (13) tells us that the output of the stream produced by  $f_{o,n}$  at  $x$  is  $o(x)$ , and Equation (14) tells us that the rest of the stream is the stream produced by  $f_{o,n}$  at  $x$  is  $f_{o,n}(n(x))$ .

**Remark I.6.** Analogous to Remark 2.13, there can only ever be *one* unfold of this type. This is because streams are a terminal object in the category of  $(O \times -)$ -coalgebras.

We can study the weight sharing induced by streams. Consider the category  $\text{Vect}$  and the coreader comonad  $\mathbb{R} \times -$  given by the output set  $\mathbb{R}$ . Fix a coalgebra  $(\mathbb{R}, \langle o, n \rangle)$ , and represent the coalgebra map with scalars  $w_o$  and  $w_n$ , denoting the output and the next state, respectively.

<sup>13</sup>An infinite sequence of outputs, isomorphic to the set  $(\mathbb{N} \rightarrow O)$  of functions from the natural numbers into  $O$ .

Then the universal coalgebra homomorphism is a linear map  $f_{o,n} : \mathbb{R} \rightarrow \text{Stream}(\mathbb{R})$  satisfying Equations (13) and (14). If we represent this as an infinite-dimensional matrix

$$\begin{bmatrix} w_1 & w_2 & w_3 & w_4 & \dots \end{bmatrix}$$

then the induced weight sharing scheme removes any degrees of freedom: the matrix is completely determined by  $w_o$  and  $w_n$ , and is of the form:

$$\begin{bmatrix} w_o & w_n w_o & w_n^2 w_o & w_n^3 w_o & \dots \end{bmatrix}$$

### I.1.2. MOORE MACHINES.

**Example I.7** (Moore machines). Let  $O$  and  $I$  be sets, thought of as sets of outputs and inputs, respectively. Consider the endofunctor  $O \times (I \rightarrow -) : \text{Set} \rightarrow \text{Set}$ . Then the set  $\text{Moore}_{O,I}$  of Moore Machines with  $O$ -labelled outputs and  $I$ -labelled inputs together with the map  $\langle \text{output}, \text{next} \rangle : \text{Moore}_{O,I} \rightarrow O \times (I \rightarrow \text{Moore}_{O,I})$  forms a coalgebra for this endofunctor. They can be represented as the following datatype.

---

```
data Moore o i = MkMoore {
  output :: o,
  nextStep :: (i -> Moore o i)
}
```

---

Like with Mealy machines, this description is coinductive. From a Moore machine we can always extract an output and a function which given an input produces another Moore machine.

In Example J.5 we will see how this will be related to general recurrent neural networks of a particular form.

**Example I.8** (Unfolds to Moore machines are terminal  $(O \times -)$ -coalgebras). Consider the endofunctor  $O \times (I \rightarrow -)$  from Example I.7, and any other  $(O \times -)$ -coalgebra  $(X, \langle o, n \rangle)$  to  $(\text{Moore}_{O,I}, \langle \text{output}, \text{next} \rangle)$ :

$$\begin{array}{ccc} X & \xrightarrow{f_{o,n}} & \text{Moore}_{O,I} \\ \langle o, n \rangle \downarrow & & \downarrow \langle \text{output}, \text{next} \rangle \\ O \times (I \rightarrow X) & \xrightarrow{O \times (I \rightarrow f_{o,n})} & O \times (I \rightarrow \text{Moore}_{O,I}) \end{array}$$

Then the map  $f_{o,n}$  is a necessarily an *unfold* to a Moore machine, a function describing how a Moore machine is obtained from a single value. It is a corecursive function as below:

---

```
f_m :: x -> Moore o i
f_m x = MkMoore (m_1 x) (\i i -> f_m (m_2 x i))
```

---

which is structural in nature, meaning it satisfies the following two equations which arise by unpacking the coalgebra homomorphism equations elementwise:

$$o(x) = \text{output}(f_{o,n}(x)) \tag{15}$$

$$f_{o,n}(n(x)(i)) = \text{next}(f_{o,n}(x))(i) \tag{16}$$

Here Equation (15) tells us that the output of the Moore machine produced by  $f_{o,n}$  at state  $x$  is given by the output of  $n$  at state  $x$ , and Equation (16) tells us that the next Moore machine produced at  $x$  and  $i$  is the one produced by  $f_{o,n}$  at  $n(x)(i)$ .

## J. Parametric 2-endofunctors and their algebras

Just like before, often at our disposal we will have a more minimal structure. Instead of requiring a strong 2-monad, we can instead require merely a strong 2-endofunctor.

The data of an algebra for a 2-endofunctor will be the same as the algebra for an endofunctor Definition 2.8.

**Example J.1** (Folding RNN cell). Consider the endofunctor  $1 + A \times -$  from Example 2.9. Via strength from Example E.9 we can form the 2-endofunctor  $\text{Para}(1 + A \times -) : \text{Para}(\text{Set}) \rightarrow \text{Para}(\text{Set})$ . Then a folding recurrent neural network arises as its algebra.

More concretely, an algebra here consists of the carrier set  $S$  (name suggestively chosen to denote *hidden state*) and a parametric map  $(P, \text{cell}^{\text{rnt}}) \in \text{Para}(\text{Set})(1 + A \times S, S)$ . Via the isomorphism  $P \times (1 + A \times S) \cong P + P \times A \times S$  we can break  $\text{cell}^{\text{rnt}}$  into two pieces: the choice of the initial hidden state  $\text{cell}_0^{\text{rnt}} : P \rightarrow S$  and the folding recurrent neural network cell  $\text{cell}_1^{\text{rnt}} : P \times A \times S \rightarrow S$ , as shown in the figure below.



Figure 5. An algebra for  $\text{Para}(1 + A \times -)$  consists of a recurrent neural network cell and an initial hidden state.

We will see how iterating this construction will produce a folding recurrent neural network which consumes a sequence of inputs and iteratively updates its hidden state.<sup>14</sup>

**Example J.2** (Recursive NN cell). Consider the endofunctor  $A + (-)^2$  from Example 2.10. Via strength from Example E.9 we can form the 2-endofunctor  $\text{Para}(A + (-)^2) : \text{Para}(\text{Set}) \rightarrow \text{Para}(\text{Set})$ . Then a recursive neural network arises as its algebra.

More concretely, an algebra here consists of the carrier set  $S$  and a parametric map  $(P, \text{cell}^{\text{rcsv}}) \in \text{Para}(\text{Set})(A + S^2, S)$ . Via the isomorphism  $P \times (A + S^2) \cong P + P \times A \times S^2$  we can break  $\text{cell}^{\text{rcsv}}$  into two pieces: the choice of the initial hidden state  $\text{cell}_0^{\text{rcsv}} : P \rightarrow S$  and the recursive neural network cell  $\text{cell}_1^{\text{rcsv}} : P \times A \times S^2 \rightarrow S$ , as shown in the figure below.



Figure 6. An algebra for  $\text{Para}(A + (-)^2)$  consists of a recursive neural network cell and an initial hidden state.

**Example J.3** (Unfolding RNN cell). Consider the endofunctor  $O \times -$  from Example I.4. Via strength  $\sigma_{P,X} : P \times (O \times X) \xrightarrow{\cong} O \times P \times X$  we can form the 2-endofunctor  $\text{Para}(O \times -) : \text{Para}(\text{Set}) \rightarrow \text{Para}(\text{Set})$ . Then an unfolding recurrent neural network arises as its coalgebra.

More concretely, a coalgebra here consists of the carrier set  $S$ , and a parametric map  $(P, \langle \text{cell}_o, \text{cell}_n \rangle) \in \text{Para}(\text{Set})(S, O \times S)$ . Here  $\langle \text{cell}_o, \text{cell}_n \rangle$  consists of maps  $\text{cell}_o : P \times S \rightarrow O$  which computes the output and  $\text{cell}_n : P \times S \rightarrow S$  which computes the next state.<sup>15</sup> We will see how iterating this construction will produce an unfolding recurrent neural network



Figure 7. An algebra for the  $\text{Para}(O \times -)$  2-endofunctor consists of an unfolding recurrent neural network cell.

given a starting state produces a stream of outputs  $O$ .

**Example J.4** (Mealy machine cell / Full RNN cell). Consider the endofunctor  $I \rightarrow O \times -$  from Example 2.11. Via

<sup>14</sup>We note here that the hidden state is allowed to depend on the parameter, something which is not possible in the usual definition of a recurrent neural network cell.

<sup>15</sup>By universal property of the product we can treat them as one cell of type  $P \times S \rightarrow O \times S$  or as two cells.

strength from Example E.9 we can form the 2-endofunctor  $\text{Para}(I \rightarrow O \times -) : \text{Para}(\text{Set}) \rightarrow \text{Para}(\text{Set})$ . Then a Mealy machine cell arises as its coalgebra.

More concretely, a coalgebra here consists of the carrier set  $S$  and a parametric map  $(P, \text{cell}^{\text{Mealy}}) \in \text{Para}(\text{Set})(S, I \rightarrow O \times S)$ . Here  $\text{cell}^{\text{Mealy}}$  can be thought of as a map  $P \times S \rightarrow I \rightarrow O \times S$ , as shown in the figure below.



Figure 8. An algebra for  $\text{Para}((I \rightarrow O \times -))$  2-endofunctor consists of an object  $S$  and a map  $f : P \times S \times I \rightarrow O \times S$  which we have taken the liberty to uncurry.

We interpret this as a full recurrent neural network which consumes a hidden state  $S$ , and input  $I$  and produces both an output  $O$  and an updated hidden state  $S$ .

This suggests that recurrent neural networks can be thought of as learnable mealy machines, a perspective seldom advocated for in the literature.

**Example J.5 (Moore machine cell).** Consider the endofunctor  $O \times (I \rightarrow -)$  from Example I.7. Via strength  $\sigma_{P,X} : P \times O \times (I \rightarrow O) \rightarrow O \times (I \rightarrow P \times X)$  defined as  $(p, o, f) \mapsto (o, \lambda i \mapsto (p, f(i)))$  we can form the 2-endofunctor  $\text{Para}(O \times (I \rightarrow -)) : \text{Para}(\text{Set}) \rightarrow \text{Para}(\text{Set})$ . Then a Moore machine cell arises as its coalgebra.

More concretely, a coalgebra here consists of the carrier set  $S$  and a parametric map  $(P, \text{cell}^{\text{Moore}}) \in \text{Para}(\text{Set})(S, O \times (I \rightarrow S))$ . Here  $\text{cell}^{\text{Moore}}$  can be thought of as a map  $P \times S \rightarrow O \times (I \rightarrow S)$ , as shown in the figure below. We can break it down into two pieces  $\text{cell}_o^{\text{Moore}} : P \times S \rightarrow O$  and  $\text{cell}_n^{\text{Moore}} : P \times S \times I \rightarrow S$ .



Figure 9. An algebra for  $\text{Para}(O \times (I \rightarrow -))$  as a parametric Moore cell. Unlike a Mealy cell, here the output  $O$  does not depend on the current input  $I$ .

## K. Unrolling neural networks via transfinite construction

We will now reap the benefits of the transfinite construction of free monads on endofunctors, and unpack the corresponding unrolling of these networks which performs the weight sharing automatically. For the purposes of space and clarity, we unpack these neural networks as lax homomorphism of algebras for 2-endofunctors and instead of for 2-monads.

**Example K.1 (Iterated Folding RNN).** Consider an algebra  $(P, \text{cell}^{\text{rnt}})$  of  $\text{Para}(1 + A \times -)$ , i.e. a folding RNN cell (Example J.1). Its unrolling is a  $\text{Para}(1 + A \times -)$ -algebra homomorphism below, where  $\Delta_P : P \rightarrow P \times P$  is the copy map.

$$\begin{array}{ccc}
 1 + A \times \text{List}(A) & \xrightarrow{\text{Para}(1 + A \times -)((P, f_{\text{rnt}}))} & 1 + A \times X \\
 \gamma([\text{Nil}, \text{Cons}]) \downarrow & \swarrow \Delta_P & \downarrow (P, \text{cell}^{\text{rnt}}) \\
 \text{List}(A) & \xrightarrow{(P, f_{\text{rnt}})} & X
 \end{array} \tag{17}$$

Here the map  $f_{\text{rnt}}$  is the parametric analogue of a *fold* (Example 2.12):

---

```

frcnt :: (p, List a) -> x
frcnt p Nil = cellrcnt p (inl ())
frcnt p (Cons a as) = cellrcnt p (inr a (frcnt p as))
    
```

---

We can see that  $f_{\text{rcnt}}$  is structurally recursive: it processes the head of the list by applying  $\text{cell}^{\text{rcnt}}$  to the parameter  $p$  and the output of the  $f_{\text{rcnt}}$  with the same parameter, and the tail of the same list.

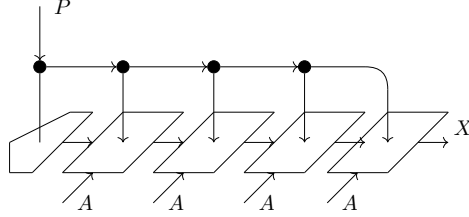


Figure 10. String diagram representation of the unrolled “folding” recurrent neural network. The weight sharing arises abstractly out of the categorical framework.

We proceed to show that  $\Delta_P$  is indeed a valid reparameterisation for these parametric morphisms. Unpacking the composition on top right side, and the bottom left side of Equation (17), respectively, yields parametric maps

$$P \times P \times (1 + A \times \text{List}(A)) \rightarrow X$$

$$P \times (1 + A \times \text{List}(A)) \rightarrow X$$

whose implementations are, respectively:

$$\begin{aligned}
 (p, q, \text{inl}(\bullet)) &\mapsto \text{cell}^{\text{rcnt}}(p, \text{inl}(\bullet)) & (p, \bullet) &\mapsto \text{cell}^{\text{rcnt}}(p, \text{inl}(\bullet)) \\
 (p, q, \text{inr}(a, as)) &\mapsto \text{cell}^{\text{rcnt}}(q, a, f(p, as)) & (p, (a, as)) &\mapsto \text{cell}^{\text{rcnt}}(p, a, f(p, as))
 \end{aligned}$$

It is easy to see that  $\Delta_P$  is a valid reparameterisation between them, “tying” the parameters  $p$  and  $q$  together.

**Example K.2** (Iterated unfolding RNN). Consider a coalgebra  $(P, \langle \text{cell}_o, \text{cell}_n \rangle)$  of  $\text{Para}(O \times -)$ , i.e. an unfolding RNN cell (Example J.3). Its unrolling is a  $\text{Para}(O \times -)$ -coalgebra homomorphism below:

$$\begin{array}{ccc}
 X & \xrightarrow{(P, f_{o,n})} & \text{Stream}(O) \\
 (P, \langle \text{cell}_o, \text{cell}_n \rangle) \downarrow & \nearrow \Delta_P & \downarrow \gamma(\langle \text{output}, \text{next} \rangle) \\
 O \times X & \xrightarrow{\text{Para}(O \times -)((P, f_{o,n}))} & O \times \text{Stream}(O)
 \end{array} \tag{18}$$

Here  $f_{o,n} : P \times X \rightarrow \text{Stream}(O)$  is the parametric analogue of an *unfold* (Example I.5):

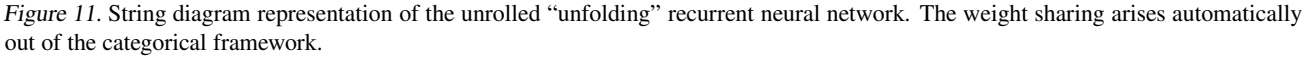
---

```

fo,n :: (p, x) -> Stream o
fo,n p x = MKStream (cello p x) (fo,n p (celln x))
    
```

---

We can see that  $f_{o,n}$  is structurally corecursive: it produces a stream whose head  $\text{cell}_o(p, x)$ , and whose tail is the stream produced by  $f_{o,n}$  at  $\text{cell}_n(p, x)$ .



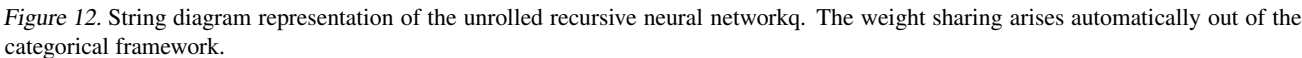
**Example K.3** (Iterated Recursive NN). Consider an algebra  $(P, \text{cell}^{\text{rcsv}})$  of  $\text{Para}(A + (-)^2)$ , i.e. a recursive neural network cell (Example J.2). Its unrolling is a  $\text{Para}(A + (-)^2)$ -algebra homomorphism below:

$$\begin{array}{ccc}
A + \text{Tree}(A)^2 & \xrightarrow{\text{Para}(A+(-)^2)((P, f_{\text{rcsv}}))} & A + X^2 \\
\cong \downarrow & \swarrow \Delta_P & \downarrow (P, \text{cell}^{\text{rcsv}}) \\
\text{Tree}(A) & \xrightarrow{(P, f_{\text{rcsv}})} & X
\end{array} \quad (19)$$

Here  $f_{\text{rcsv}}$  is a function with the following implementation

```
frCSV :: (p, Tree a) -> x
frCSV p (Leaf a) = cellrCSV p (inl a)
frCSV p (Node l r) = cellrCSV p inr (f (frCSV p l) (frCSV p r))
```

This function too performs weight sharing, as the recursive call to subtrees is done with the same parameter  $p$ .



**Example K.4** (Iterated Mealy machine cell). Consider an algebra  $(P, \text{cell}^{\text{Mealy}})$  of  $\text{Para}(I \rightarrow O \times -)$ , i.e. a Mealy machine cell (Example 2.11). Its unrolling is a  $\text{Para}(I \rightarrow O \times -)$ -algebra homomorphism below:

$$\begin{array}{ccc}
X & \xrightarrow{(P, f_n)} & \mathbf{Mealy}_{O,I} \\
(P,n) \downarrow & \nearrow \Delta_P & \downarrow \gamma(\text{next}) \\
(I \rightarrow O \times X) & \xrightarrow{\text{Para}(I \rightarrow O \times -)((P, f_n))} & (I \rightarrow O \times \mathbf{Mealy}_{O,I})
\end{array} \quad (20)$$

Here  $f_n$  is a corecursive function with the following implementation



---

```

f_n :: (p, x) -> Mealy o i
f_n (p, x) = MkMealy $ \i -> let (o', x') = n p x i
                                in (o', f_n p x')
    
```

---

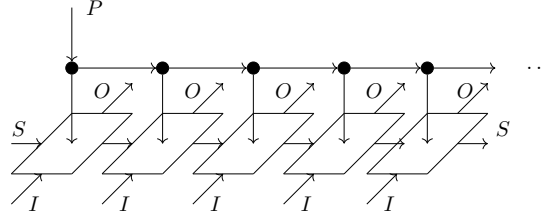


Figure 13. String diagram representation of a parametric mealy machine — a recurrent neural network.

**Example K.5** (Iterated Moore machine cell). Consider a coalgebra  $(P, \text{cell}^{\text{Moore}})$  of  $\text{Para}(O \times (I \rightarrow -))$ , i.e. a Moore machine cell (Example I.7). Its unrolling is a  $\text{Para}(O \times (I \rightarrow -))$ -coalgebra homomorphism below:

$$\begin{array}{ccc}
 X & \xrightarrow{(P, f_n)} & \text{Moore}_{O, I} \\
 (P, \langle o, n \rangle) \downarrow & \nearrow \Delta_P & \downarrow \gamma(\text{next}) \\
 O \times (I \rightarrow X) & \xrightarrow[\text{Para}(O \times (I \rightarrow -))((P, f_n))]{\quad} & (I \rightarrow O \times \text{Moore}_{O, I})
 \end{array} \tag{21}$$

In code,  $f_n$  is a corecursive function

---

```

f_n :: (p, x) -> Moore o i
f_n (p, x) = MkMoore $ \i -> (o p x, f_n p (n p x i))
    
```

---

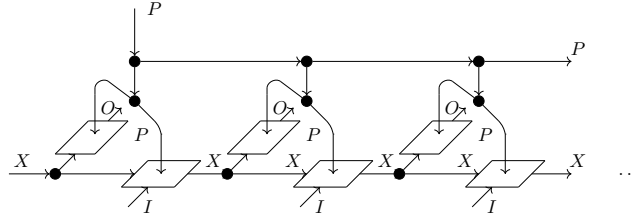


Figure 14. String diagram representation of an unrolled Moore machine.